

Machine Learning &
Predictive Analytics

BANK TERM DEPOSIT PREDICTIONS

Presentation by Eat & Drink



AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

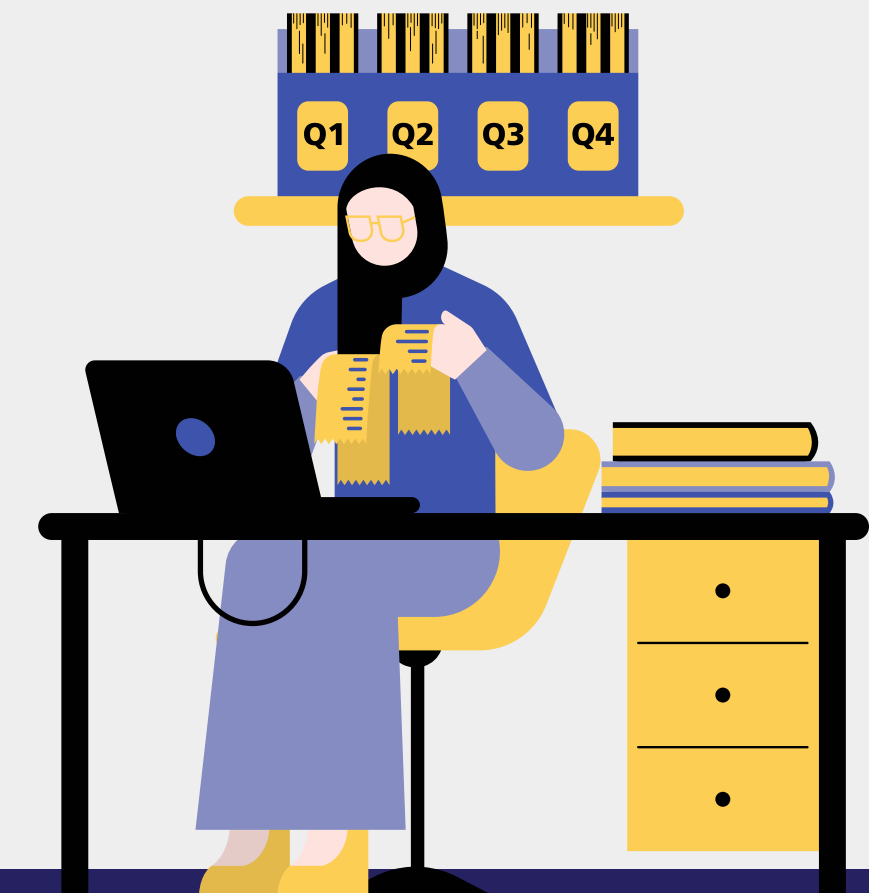
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset



TARGET OF DATASET

การวิเคราะห์แคมเปญการตลาดทางตรง เพื่อสร้างโมเดลทำนายการสมัคร
บัญชีเงินฝากประจำ (Term Deposit)
โดยศึกษาความสัมพันธ์ของข้อมูลคุณลักษณะลูกค้าและประวัติการติดต่อ
เพื่อค้นหาโอกาสความสำเร็จในการนำเสนอผลิตภัณฑ์ และปรับปรุงกลยุทธ์
การตลาดให้แม่นยำยิ่งขึ้น

โดยพิจารณาปัจจัยสำคัญ เช่น

- ข้อมูลส่วนบุคคล: อายุ, อาชีพ, การศึกษา
- สถานะทางการเงิน: ยอดเงินในบัญชี, สถานะสินเชื่อ
- รายละเอียดการติดต่อ: ระยะเวลาสนทนา, ผลลัพธ์จากแคมเปญก่อนหน้านี้



AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

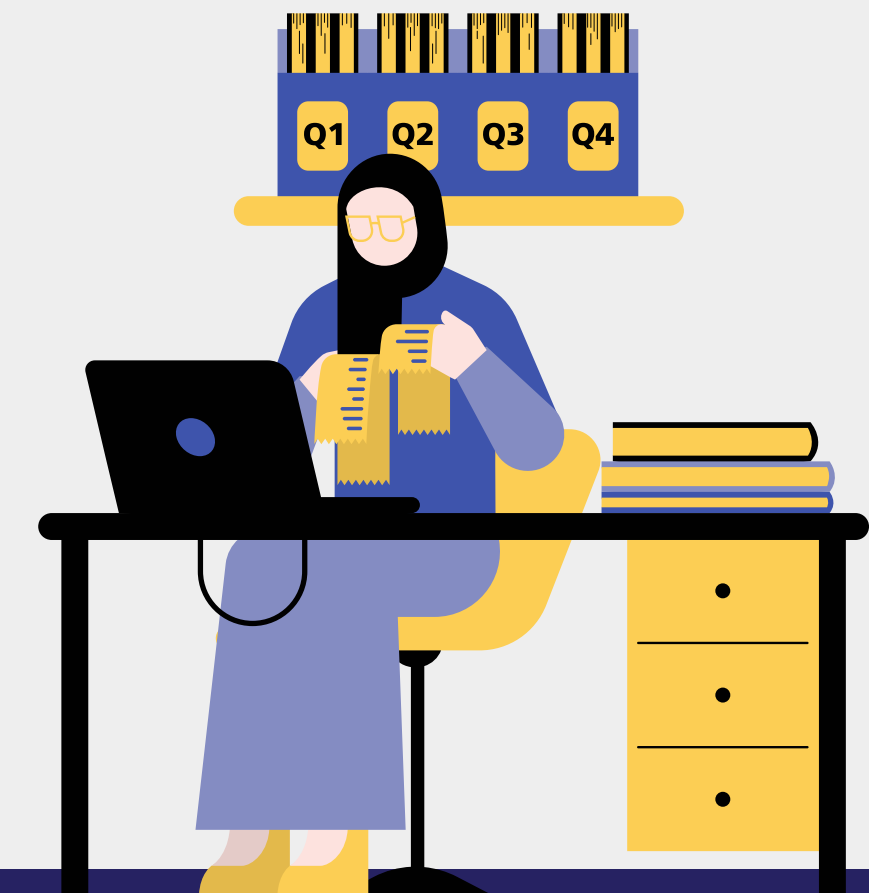
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset



● Pain point

1. อัตราการสมัคร Term deposit ต่ำมาก
 - จากแคมเปญ marketing ผ่านโทรศัพท์ ส่วนใหญ่ลูกค้าไม่สมัคร
 - ฐานข้อมูลต้นฉบับพบว่า ลูกค้าที่สมัครจริง (Yes) มีน้อยกว่า 15%
2. สิ้นเปลืองต้นทุนทางการตลาด
 - โทรหาลูกค้าจำนวนมาก → แต่ผลลัพธ์เฉพาะลูกค้าบางกลุ่มเท่านั้นที่สนใจ
 - ทีมต้องการ “Target ลูกค้าที่ใช้” ไม่ใช่โทรสุ่ม
 - ในเชิงธุรกิจ การพลาดลูกค้าที่มีโอกาสสมัครจริง (False Negative) มีต้นทุนสูงกว่าการติดต่อเกินความจำเป็น (False Positive)
3. ข้อมูลลูกค้าหลากหลาย → วางแผน segmentation ยาก
 - เช่น อายุ, อาชีพ, สถานะการเงิน, ประวัติการติดต่อ
 - ข้อมูลเยอะแต่ไม่รู้ว่าคุณสมบัติไหนส่งผลต่อการตัดสินใจสมัครมากที่สุด

● Motivation

- ใช้ Machine Learning เพื่อช่วยเลือกกลุ่มลูกค้าที่มีโอกาสสมัครจริง
- ลดต้นทุน + เพิ่ม conversion rate ของแคมเปญ
- ใช้วิเคราะห์พฤติกรรมลูกค้าเชิงลึก เพื่อปรับกลยุทธ์การตลาดให้มีประสิทธิภาพ





• Challenges

- Class Imbalance สูงมาก: "No" เยอะกว่า "Yes" มาก
- ข้อมูลแบบ categorical จำนวนมาก ต้อง encode ให้ถูกต้อง
- Feature จำนวนมาก → ต้องเลือกฟีเจอร์ที่สำคัญจริง
- Distribution ไม่สม่ำเสมอ เช่น duration, campaign contact จำนวนครั้งไม่ปกติ

• Benefits

- คัดกรองแม่นยำ: โฟกัสเฉพาะกลุ่มที่มีโอกาสสมัครสูง
- สร้างโมเดลเพื่อนำไปใช้ในการคัดเลือกลูกค้าเป้าหมาย(Customer targeting)
- ช่วยวาง Strategy ให้ธนาคารในอนาคต เช่น เลือกกลุ่มอายุเหมาะสม / ลดจำนวนครั้งในการโทร / เน้นช่วงเวลาเฉพาะ
- ลดต้นทุน : เพิ่มประสิทธิภาพการใช้เวลาละงบประมาณของทีม
- เพิ่มโอกาสในการขายและสร้างความพึงพอใจให้ลูกค้า



AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

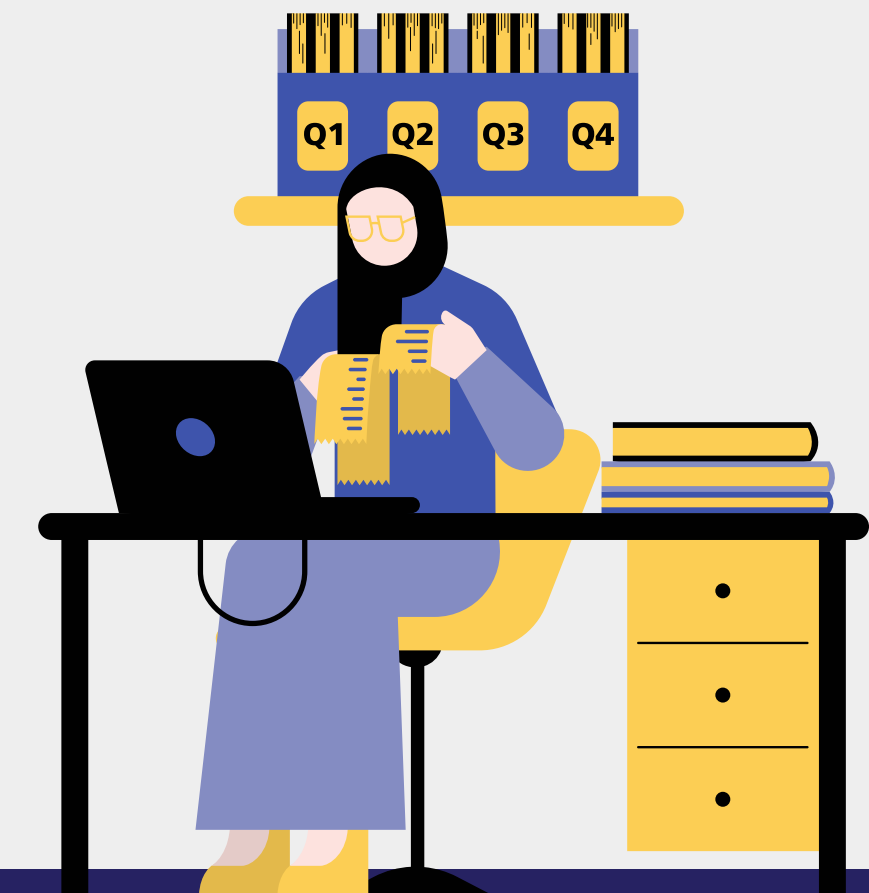
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset



BANK MARKETING DATASET



45,211
รายการข้อมูลลูกค้า

ตัวแปรต้น (Attributes)

- Age : อายุลูกค้า
- Job : อาชีพ
- Balance : ยอดเงินคงเหลือในบัญชี
- Duration : ระยะเวลาที่ใช้ในการสนทนาโทรศัพท์

ตัวแปรเป้าหมาย (Target Variable)

y (Target) : ลูกค้าต้องรับ (yes) หรือปฏิเสธ (no)

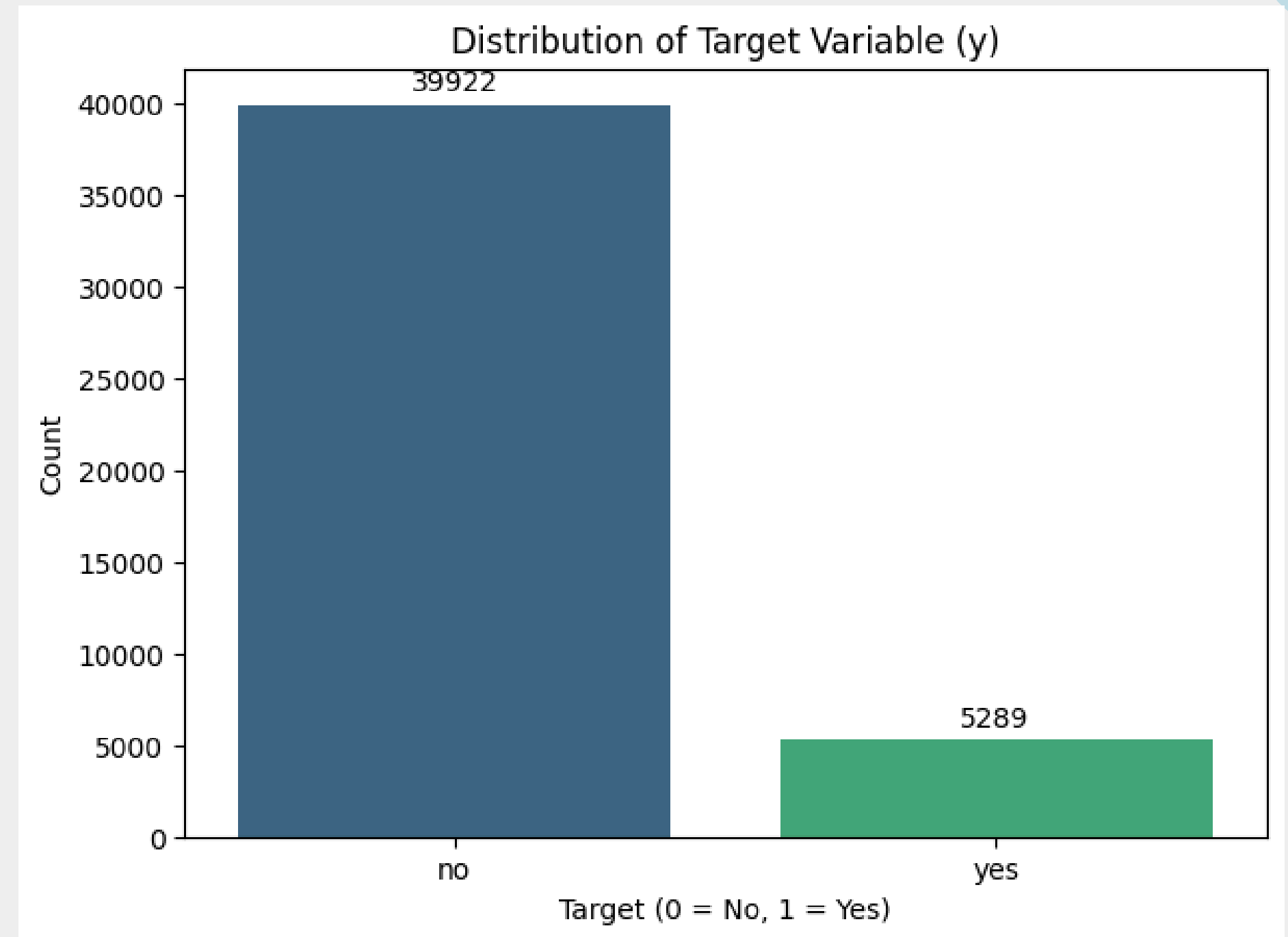
Age	Job	Balance	Duration	y
35	management	1500	200	no
42	technician	560	350	yes
28	entrepreneur	-100	120	no
51	blue-collar	2300	450	yes

DATA EXPLORATION

```
#การสำรวจข้อมูล (Data Exploration)
print("\n--- Target Distribution ---")
plt.figure(figsize=(7, 5))
ax = sns.countplot(x='y', data=df, palette='viridis')
for container in ax.containers:
    ax.bar_label(container, fmt='%d', padding=3)

plt.title('Distribution of Target Variable (y)')
plt.ylabel('Count')
plt.xlabel('Target (0 = No, 1 = Yes)')
plt.show()
```

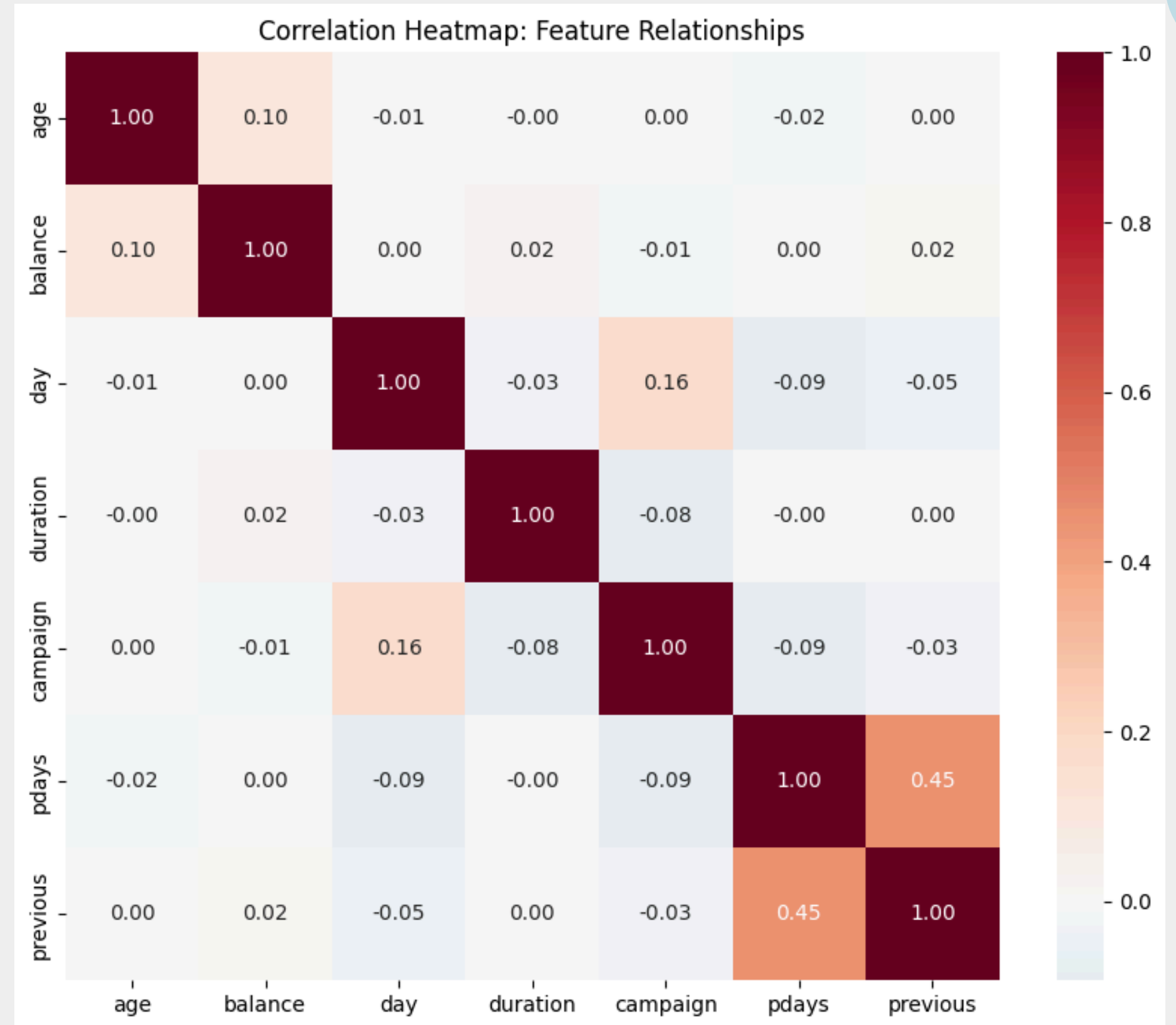
จากการสำรวจข้อมูลเบื้องต้น จะเห็นว่ามีความไม่สมดุล เพราะสัดส่วนในการตอบรับ “Yes” น้อยมาก นั่นแสดงว่า โมเดลเรียนรู้ที่จะตอบ “No”



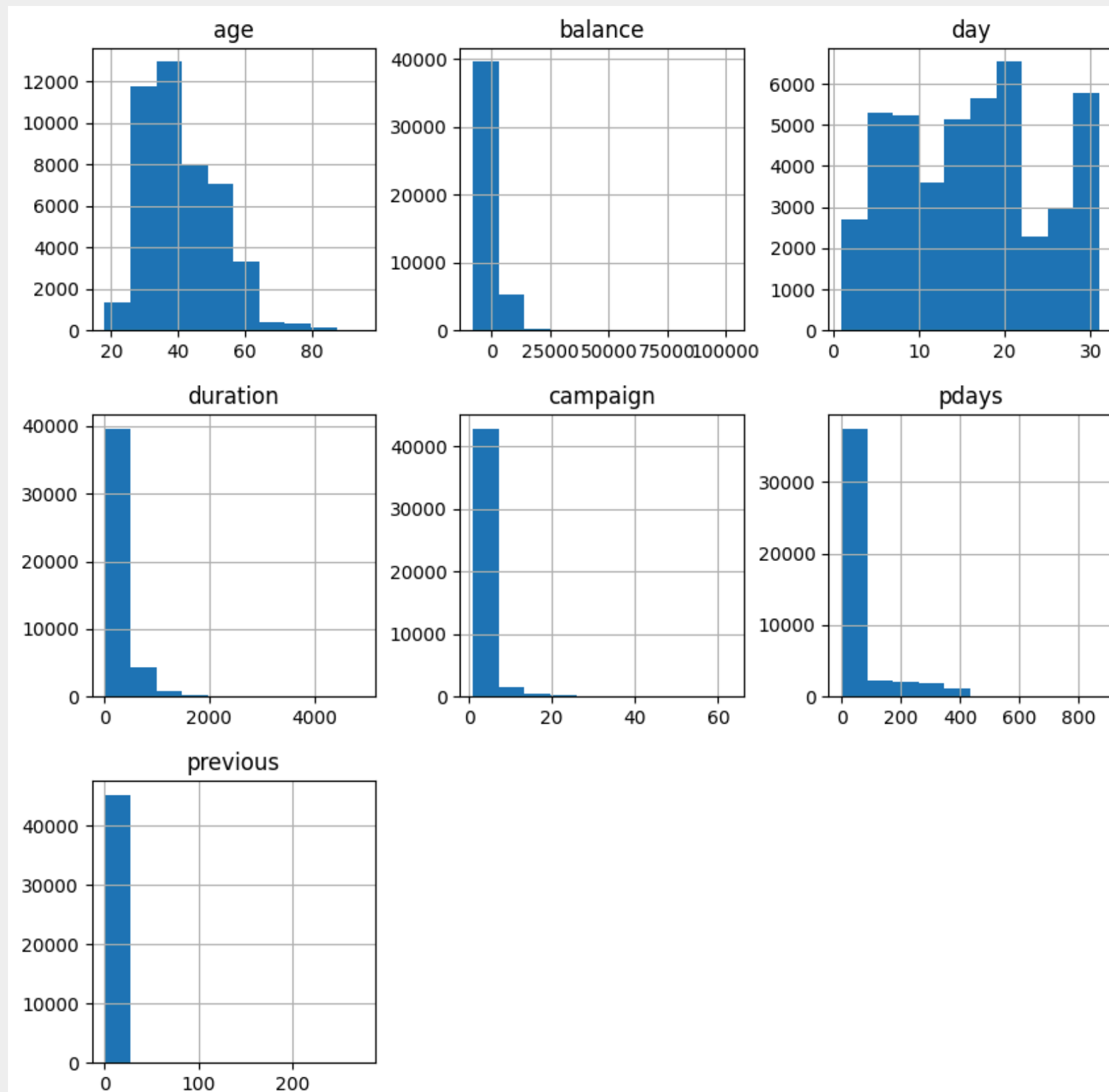
วิเคราะห์ความสัมพันธ์ - CORRELATION HEATMAP

```
#วิเคราะห์ความสัมพันธ์เชิงเส้น (Heatmap) ---
plt.figure(figsize=(10, 8))
numeric_df = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=True,
            cmap='RdBu_r', fmt='.2f', center=0)
plt.title('Correlation Heatmap: Feature Relationships')
plt.show()
```

- **pdays – previous** มีค่า 0.45 หมายถึง ความสัมพันธ์เชิงบวกปานกลาง: คนที่ถูกติดต่อมาก่อน (previous) มักมี pdays (จำนวนวันที่ผ่านไป) ที่สัมพันธ์กัน
- **campaign – day** มีค่า 0.16 หมายถึง ความสัมพันธ์เชิงบวกอ่อน ๆ: วันที่ติดต่ออาจมีผลต่อจำนวนครั้งที่ติดต่อ
- **balance – age** มีค่า 0.10 หมายถึง ความสัมพันธ์เชิงบวกอ่อนมาก: คนอายุมากขึ้นอาจมี balance มากขึ้นเล็กน้อย



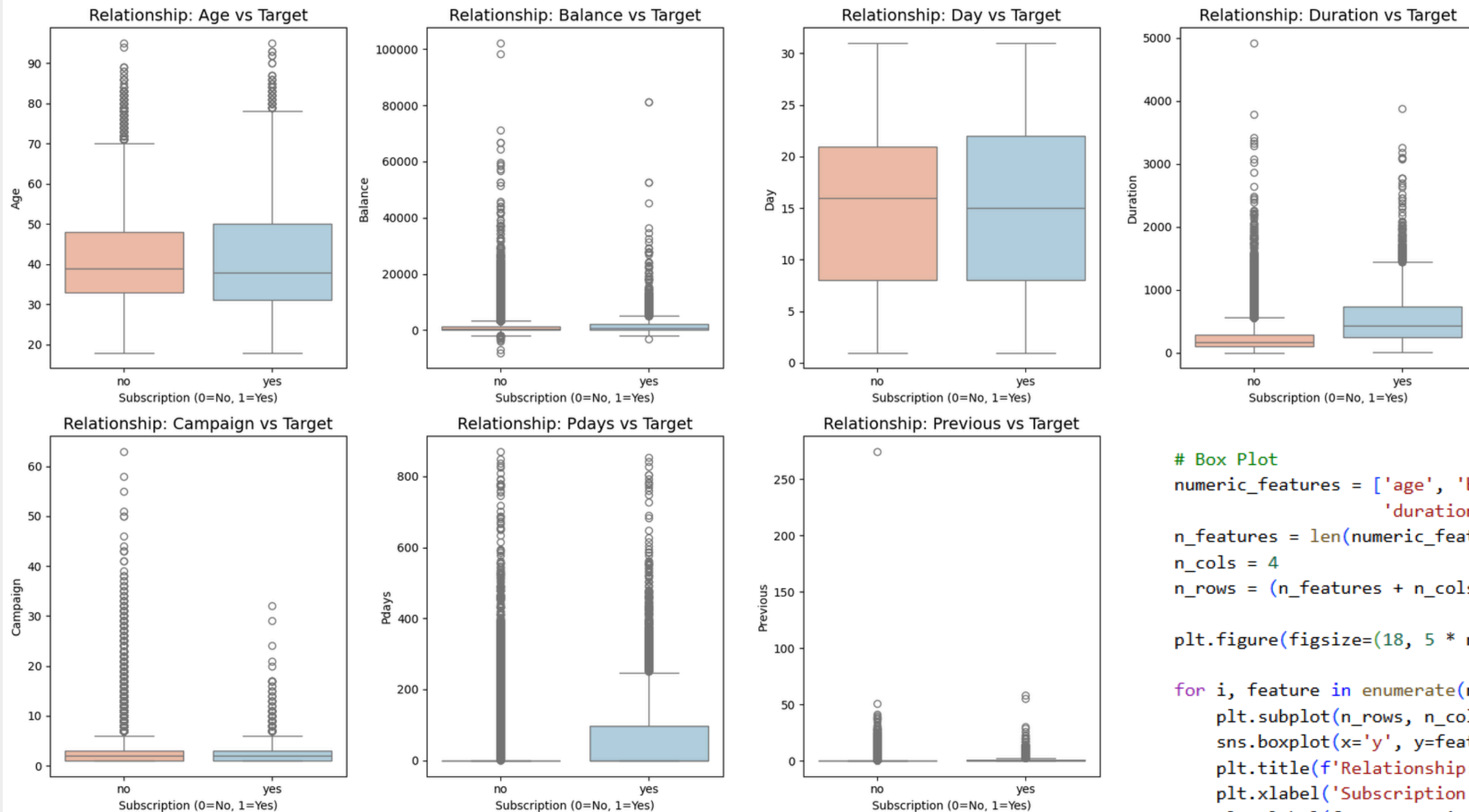
วิเคราะห์ความสัมพันธ์ - FEATURE DISTRIBUTION



```
df.hist(figsize = (10,10))  
plt.show()
```

ตัวแปรที่มีนัยสำคัญต่อการสมัครคือ duration, balance, previous, pdays และ campaign เพราะทั้งห้าตัวนี้แสดงความแตกต่างชัดเจนระหว่างคนที่สมัครกับไม่สมัคร โดยเฉพาะ duration ที่เด่นที่สุด ส่วน previous และ pdays สอดคล้องกับ heatmap ที่พบว่ามีความสัมพันธ์กันระดับปานกลาง ซึ่งสะท้อนพฤติกรรมการติดต่อก่อนหน้าได้ดี

วิเคราะห์ความสัมพันธ์ - BOX PLOT



Duration: สังเกตว่ากล่องของกลุ่ม '1' (Yes) จะอยู่สูงกว่ากลุ่ม '0' (No) ชัดเจน ยืนยันว่า Duration เป็น ปัจจัยหลัก (Key Driver)

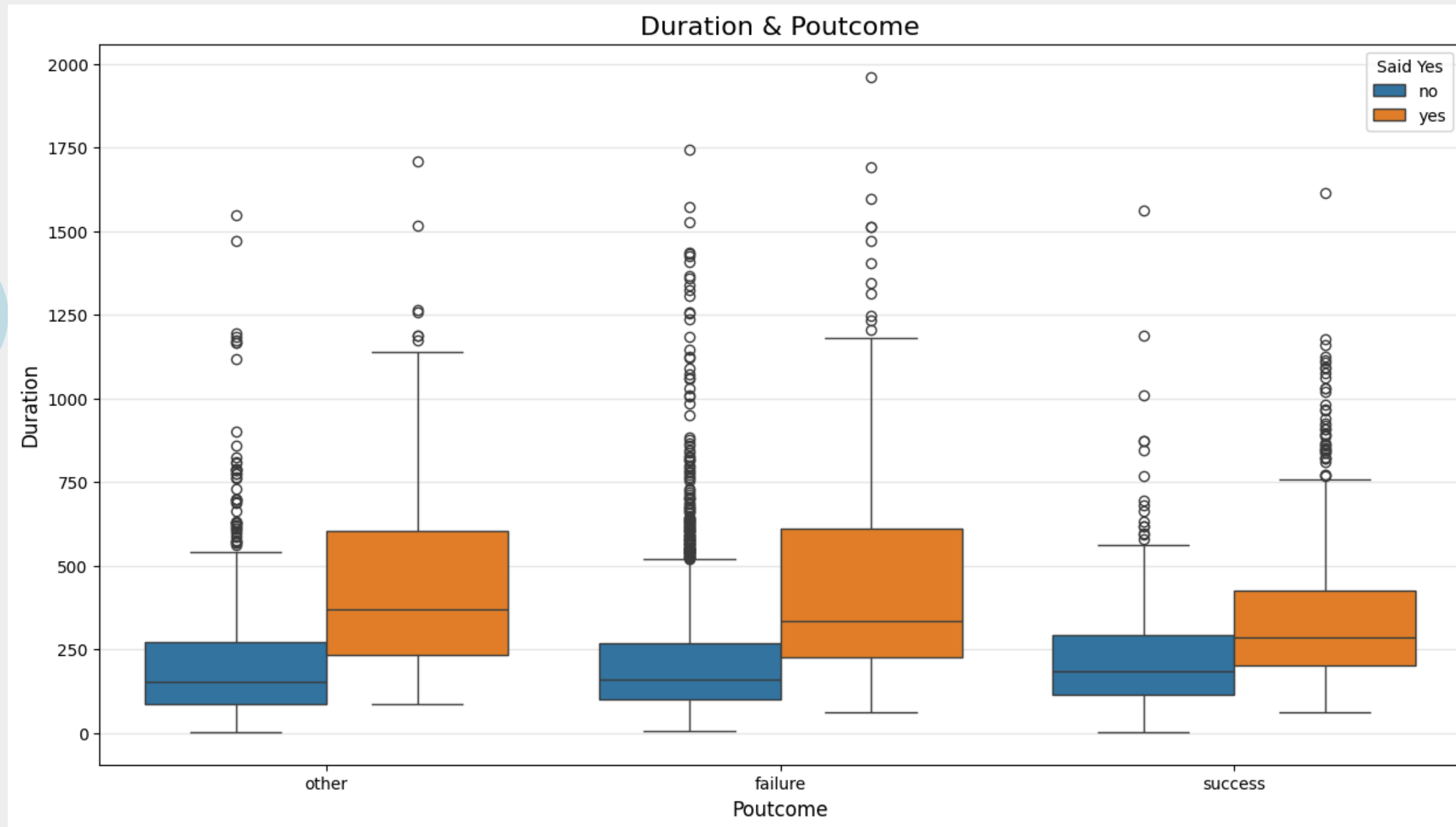
```
# Box Plot
numeric_features = ['age', 'balance', 'day',
                    'duration', 'campaign', 'pdays', 'previous']
n_features = len(numeric_features)
n_cols = 4
n_rows = (n_features + n_cols - 1) // n_cols

plt.figure(figsize=(18, 5 * n_rows))

for i, feature in enumerate(numeric_features):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.boxplot(x='y', y=feature, data=df, palette='RdBu')
    plt.title(f'Relationship: {feature.capitalize()} vs Target', fontsize=14)
    plt.xlabel('Subscription (0=No, 1=Yes)')
    plt.ylabel(feature.capitalize())

plt.tight_layout()
plt.show()
```

วิเคราะห์ความสัมพันธ์: ระยะเวลาคุยกับผลครึ่งเก่า



กลุ่ม "Success" (ลูกค้าเก่าที่เคยสมัครสำเร็จ)

ลูกค้าที่เคยเชื่อใจธนาคารอยู่แล้ว "ตัดสินใจง่าย" ไม่ต้องอธิบายเยอะ แค่เสนอโปรโมชั่นที่เหมาะสม พวกเขา ก็พร้อมจะสมัครต่อทันที (มีค่ามัธยฐาน (Median) ของ Duration ต่ำกว่า กลุ่มอื่นๆ)

กลุ่ม "Failure" (ลูกค้าเก่าที่เคยปฏิเสธ)

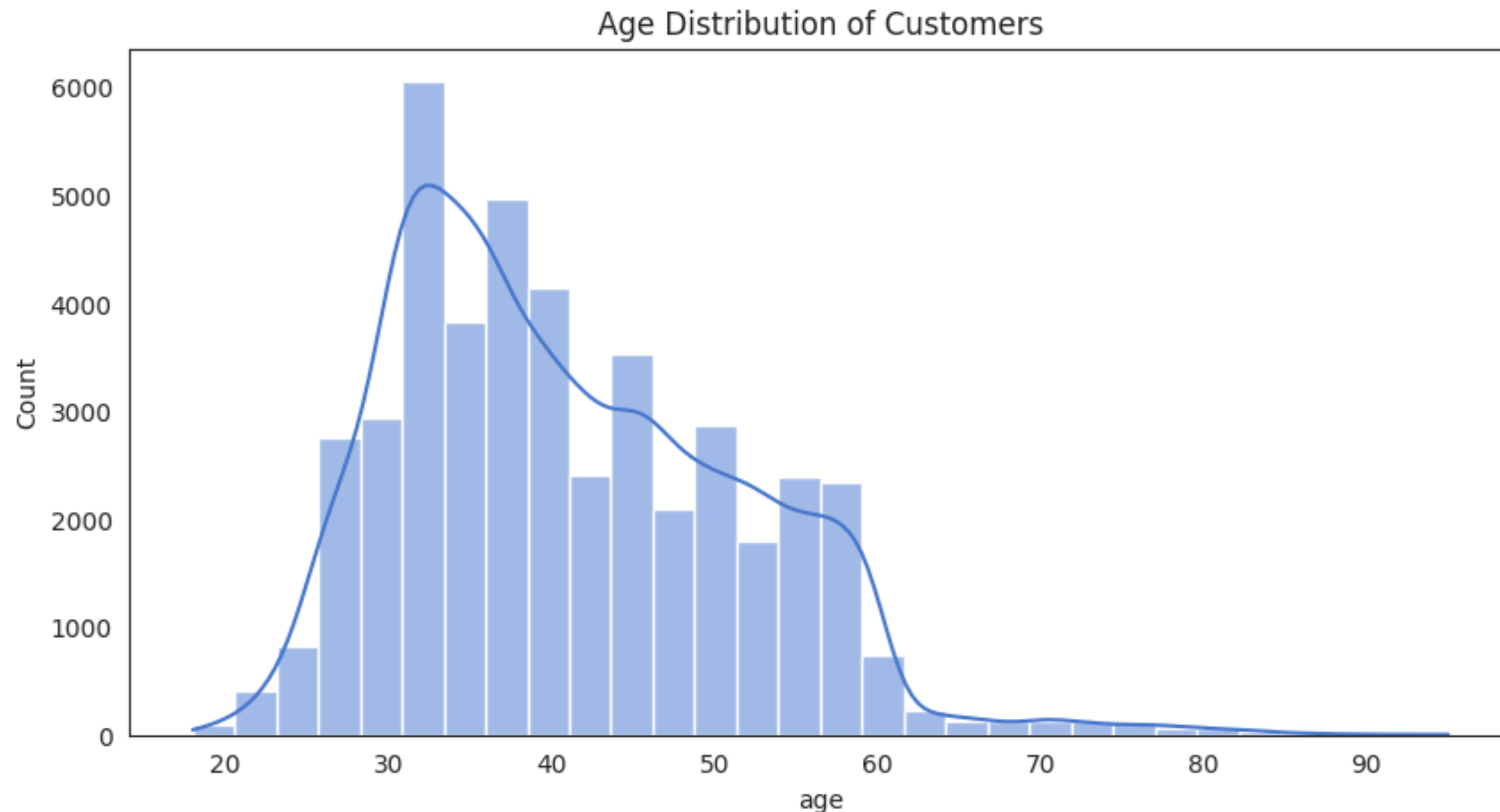
ลูกค้าที่เคยปฏิเสธไปแล้วมีความ "ลังเล" หรือมีข้อโต้แย้งในใจ พนักงานต้องใช้ความพยายามและเวลาในการโน้มน้าวใจสูงกว่าปกติมากเพื่อเปลี่ยนใจเขา

กลุ่ม "Other" (ลูกค้าใหม่ที่ไม่เคยติดต่อ)

กลุ่มลูกค้าใหม่มีความหลากหลายสูง บางคนคุยแป๊บเดียวสมัคร บางคนคุยนานก็ไม่สมัคร เป็นกลุ่มที่ทำนายยากที่สุด

วิเคราะห์ความสัมพันธ์: การกระจายตัวของอายุ

```
plt.figure(figsize=(10,5))
sns.histplot(df['age'], bins=30, kde=True,)
plt.title('Age Distribution of Customers')
plt.show()
```



กราฟแสดงการกระจายตัวแบบเบ้ขวา (Right-Skewed) โดยลูกค้ากลุ่มใหญ่ที่สุด คือ วัยทำงานช่วงอายุ 30-40 ปี ซึ่งเป็นกลุ่มที่มีกำลังซื้อสูง จึงควรเป็นกลุ่มเป้าหมายหลักในการทำตลาดแคมเปญนี้

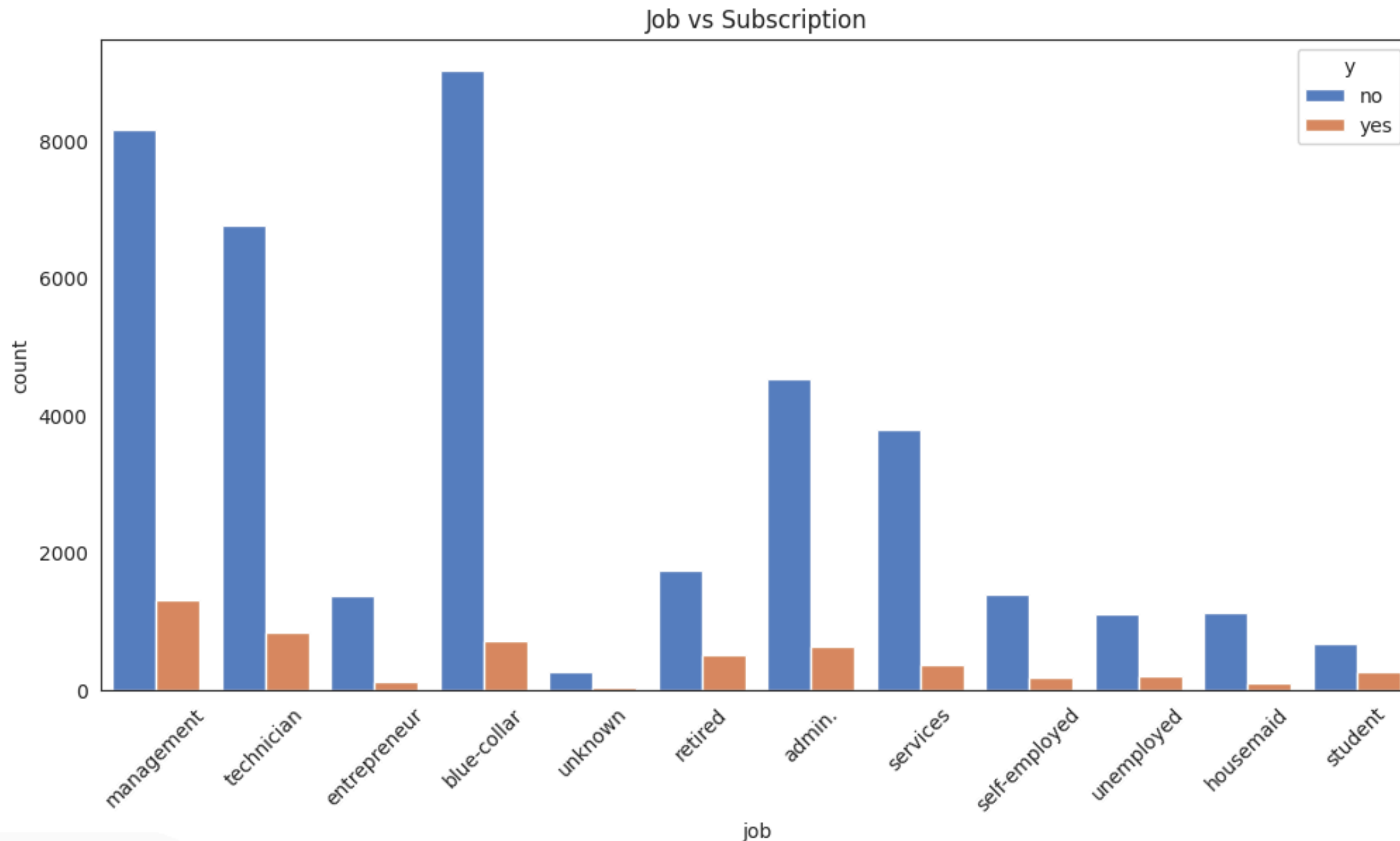
กลุ่มหลัก: ลูกค้าส่วนใหญ่มักจะกระจุกตัวอยู่ที่อายุ 30-40 ปี (วัยทำงาน)

แนวโน้ม: สัดส่วนลูกค้าลดน้อยลงเมื่อเข้าสู่วัยเกษียณ (กราฟเบ้ขวา)

Insight: ควรโฟกัสการขายผลิตภัณฑ์ที่เน้นความมั่นคงให้กลุ่มอายุ 30-40 ปี เป็นหลัก

วิเคราะห์ความสัมพันธ์ : อาชีพกับการตัดสินใจ

```
plt.figure(figsize=(12,6))
sns.countplot(x='job', hue='y', data=df)
plt.xticks(rotation=45)
plt.title('Job vs Subscription')
plt.show()
```



กลุ่มที่มี "จำนวน" การสมัครสูงสุด (Volume)

กลุ่มอาชีพ Management (ผู้บริหาร) และ Technicians (ช่างเทคนิค) มักจะเป็นกลุ่มที่มีจำนวนคนตอบ "Yes" สูงที่สุดในเชิงปริมาณ

- เนื่องจากเป็นกลุ่มฐานลูกค้าขนาดใหญ่ที่สุดใน Dataset และมีรายได้ที่มั่นคง จึงเป็นเป้าหมายหลักของธนาคาร

กลุ่มที่มี "อัตราส่วน" การสมัครสูงสุด (Success Rate)

Students (นักเรียน) และ Retired (ผู้เกษียณ): แม้จะเป็นกลุ่มที่มีจำนวนคนน้อย (แท่งกราฟเตี้ย) แต่ถ้าดูสัดส่วนคนตอบ Yes เทียบกับคนตอบ No จะพบว่ากลุ่มนี้มีอัตราการสมัครสูงกว่าอาชีพอื่น

กลุ่มที่ "เจาะยาก" (Low Conversion)

Blue-collar (พนักงานระดับปฏิบัติการ): มักจะเป็นกลุ่มที่มีจำนวนคนเยอะ แต่สัดส่วนการสมัครกลับต่ำกว่าค่าเฉลี่ย

วิเคราะห์ความสัมพันธ์ : อาชีพกับ SUCCESS RATE (%)

```
job_success_rate = pd.crosstab(df['job'], df['y'], normalize='index') * 100
job_success_rate = job_success_rate.sort_values(by='yes', ascending=False)

plt.figure(figsize=(12, 6))
sns.barplot(x=job_success_rate.index, y=job_success_rate['yes'])

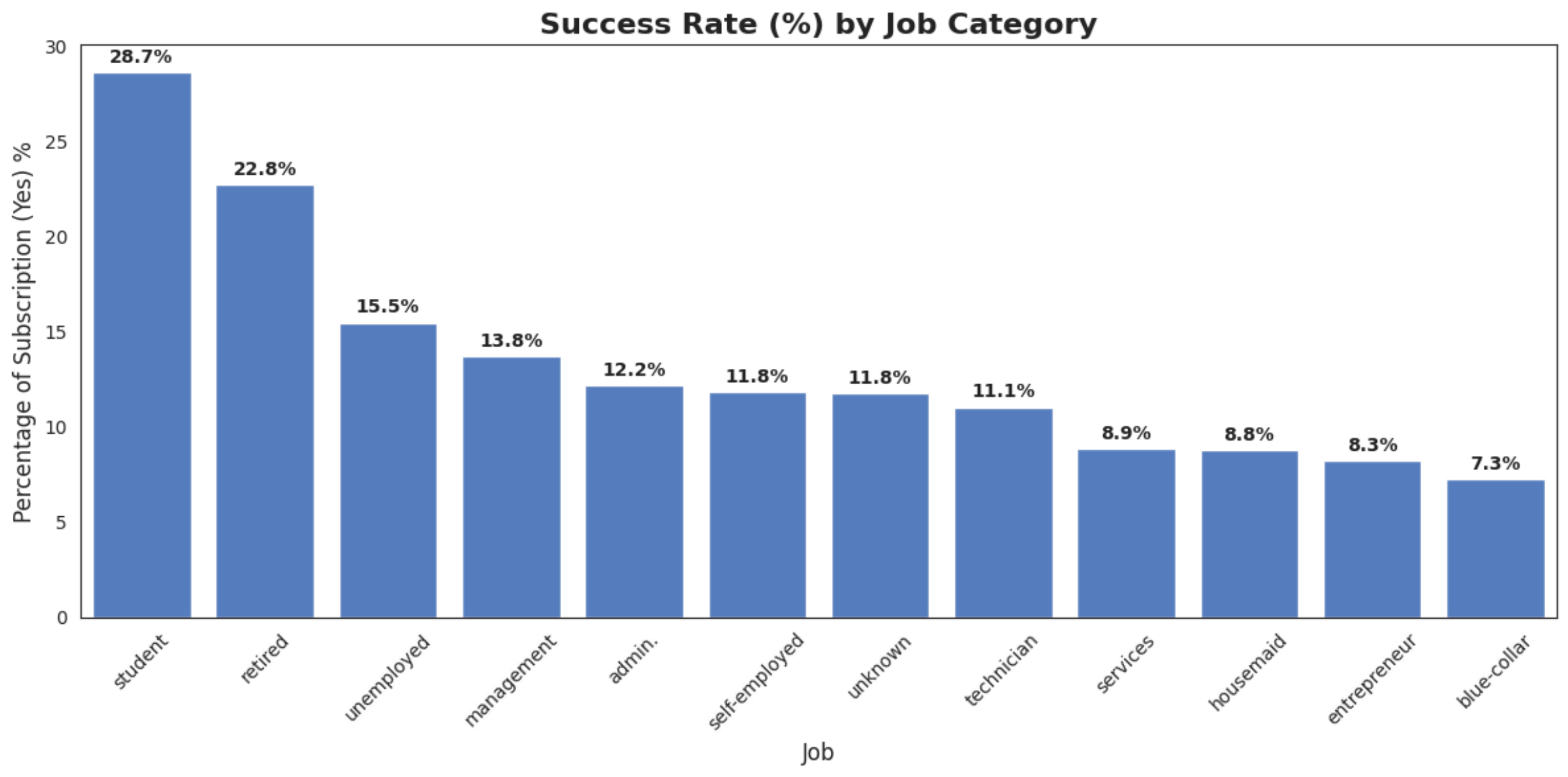
plt.title('Success Rate (%) by Job Category', fontsize=16, fontweight='bold')
plt.ylabel('Percentage of Subscription (Yes) %', fontsize=12)
plt.xlabel('Job', fontsize=12)
plt.xticks(rotation=45)

for i, rate in enumerate(job_success_rate['yes']):
    plt.text(i, rate + 0.5, f'{rate:.1f}%', ha='center', fontweight='bold')

plt.tight_layout()
plt.show()

# 4. แสดงตารางสรุปตัวเลข
print("ตารางสรุปอัตราความสำเร็จ (%) รายอาชีพ:")
print(job_success_rate['yes'].round(2))
```

ตารางสรุปอัตราความสำเร็จ (%) รายอาชีพ:	
job	
student	28.68
retired	22.79
unemployed	15.50
management	13.76
admin.	12.20
self-employed	11.84
unknown	11.81
technician	11.06
services	8.88
housemaid	8.79
entrepreneur	8.27
blue-collar	7.27
Name: yes, dtype: float64	



Student (28.7%) และ Retired (22.8%) มีอัตราการตอบรับสูงที่สุด ซึ่งสูงกว่าค่าเฉลี่ยของอาชีพอื่นๆ เกือบ 2 เท่า
ธนาคารควรปรับกลยุทธ์โดยเน้นกลุ่ม "นักเรียน" และ "ผู้เกษียณ" เป็นลำดับต้นๆ เพราะโทรหาแล้วมีโอกาสปิดการขาย (Conversion Rate) ได้สูงที่สุด

วิเคราะห์ความสัมพันธ์ : อาชีพและเงินในบัญชี

```
job_balance = df.groupby('job')['balance'].mean().sort_values(ascending=False)

plt.figure(figsize=(12, 6))
sns.barplot(x=job_balance.index, y=job_balance.values,)

plt.title('Average Account Balance by Job Category', fontsize=16, fontweight='bold')
plt.ylabel('Average Balance', fontsize=12)
plt.xlabel('Job Type', fontsize=12)
plt.xticks(rotation=45)

for i, balance in enumerate(job_balance.values):
    plt.text(i, balance + 20, f'{int(balance):,}', ha='center', fontweight='bold')

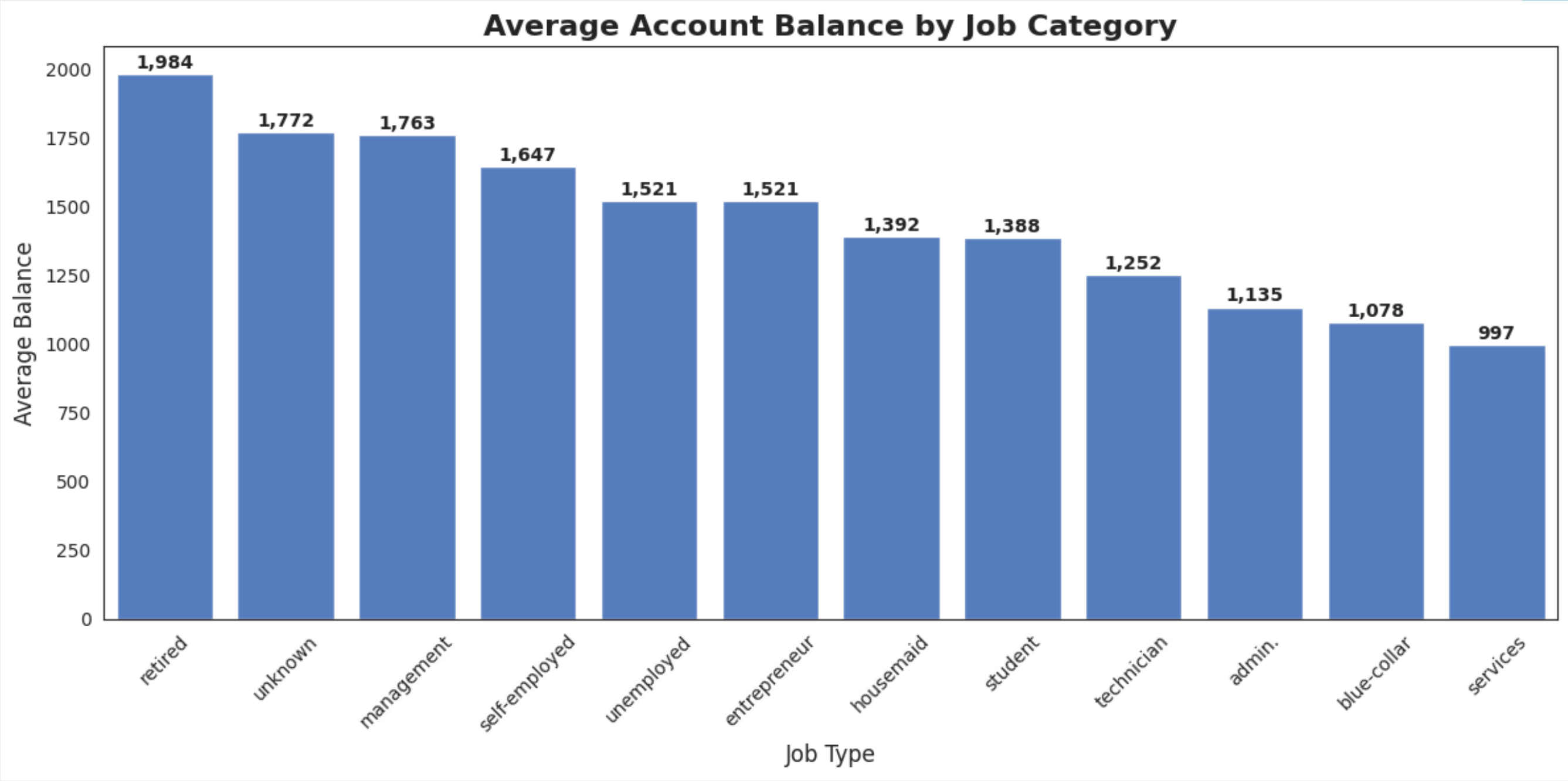
plt.tight_layout()
plt.show()

print("ตารางสรุปค่าเฉลี่ยเงินในบัญชีรายอาชีพ :")
print(job_balance.map('{:, .2f}'.format))
```

ตารางสรุปค่าเฉลี่ยเงินในบัญชีรายอาชีพ :

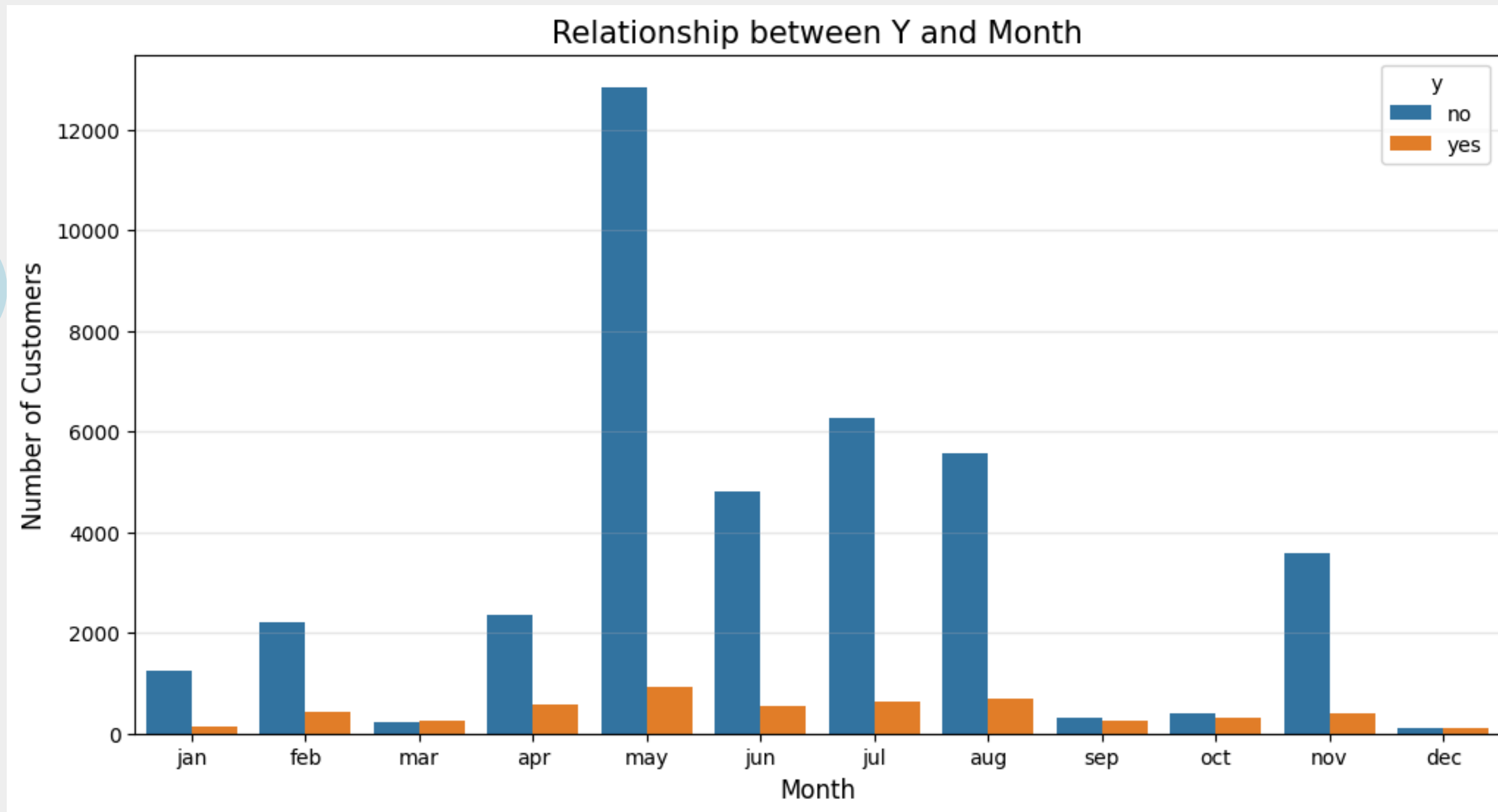
job	
retired	1,984.22
unknown	1,772.36
management	1,763.62
self-employed	1,647.97
unemployed	1,521.75
entrepreneur	1,521.47
housemaid	1,392.40
student	1,388.06
technician	1,252.63
admin.	1,135.84
blue-collar	1,078.83
services	997.09

Name: balance, dtype: object



กลุ่ม Retired (ผู้เกษียณ) และ Management (ผู้บริหาร) มักจะมีค่าเฉลี่ยเงินในบัญชีสูงที่สุด
กลุ่มนี้มี "เงินเย็น" มากพอที่จะเปลี่ยนจากเงินฝากออมทรัพย์ทั่วไปมาเป็น "เงินฝากประจำ (Term Deposit)" เพื่อรับดอกเบี้ยที่สูงกว่า
ซึ่ง "Balance" เป็นตัวแปรสำคัญ เพราะคนที่มียอดเงินในบัญชีสูง มีแนวโน้มทางสถิติที่จะตอบรับการฝากประจำมากกว่า

วิเคราะห์ความสัมพันธ์: เดือนที่ติดต่อหาลูกค้า



กราฟแสดงจำนวนการปฏิเสธแยก
รายเดือน พบว่า เดือนพฤษภาคม
(May) มีจำนวนลูกค้าปฏิเสธสูงที่สุด
อย่างมีนัยสำคัญ ตามด้วยช่วงกลาง
ปี (Jun-Aug) ในขณะที่ช่วงปลายปีมี
อัตราการปฏิเสธต่ำกว่า

- **Critical Period:** เดือน
พฤษภาคม (May) คือช่วงที่
ลูกค้าปฏิเสธเยอะที่สุด (Peak
Rejection)
- **Trend:** การปฏิเสธจะกระจุกตัว
อยู่ในช่วงกลางปี (May-Aug)

AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

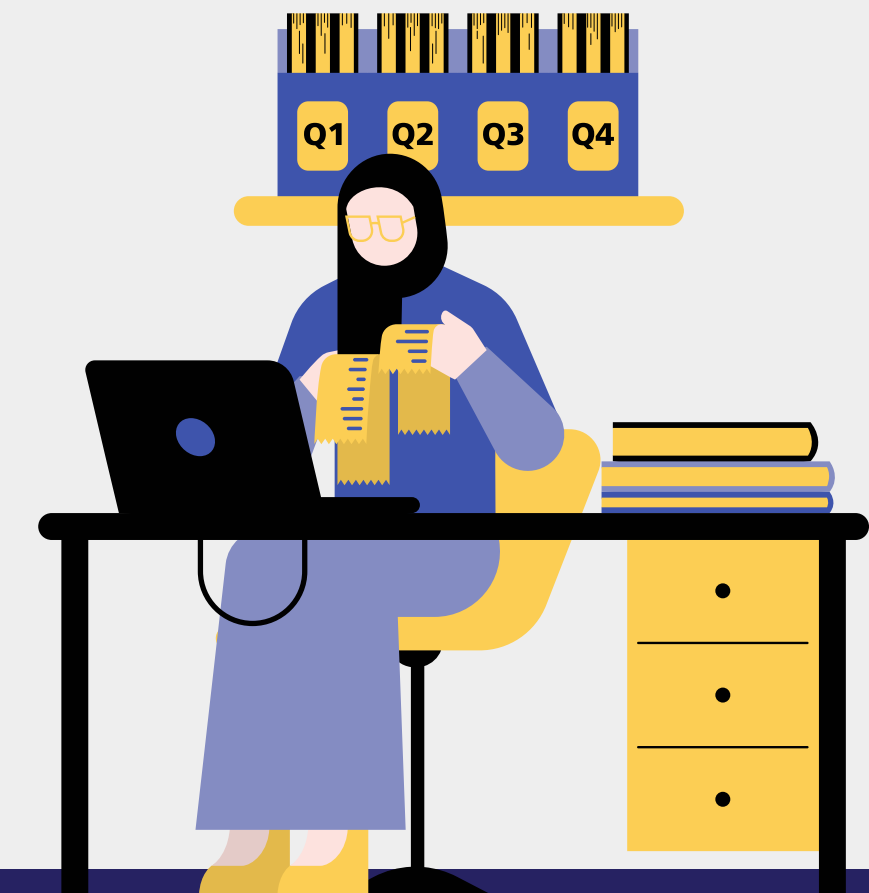
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset



การเตรียมข้อมูล : แปลง 'คำ' ให้เป็น 'ตัวเลข'

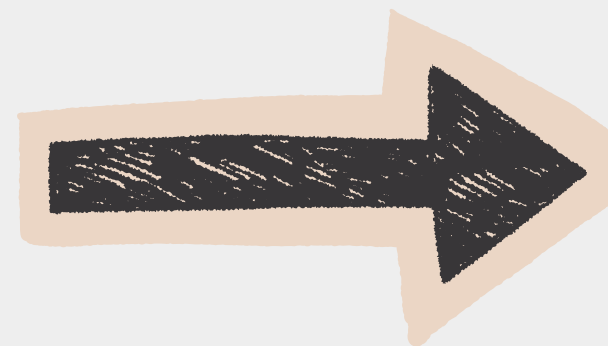
เนื่องจาก Machine Learning ไม่สามารถประมวลผลข้อมูลที่เป็นข้อความโดยตรงจึงจำเป็นต้องแปลงข้อมูลก่อน

- **Label Encoding** : แปลงตัวแปรเป้าหมาย 'y' (Target) จากคำว่า 'yes' หรือ 'no' แทนด้วย $y = 1$ และ $no = 0$
- **One-Hot-Encoding** : แปลงตัวแปรอิสระที่เป็นกลุ่มคำ เช่น สถานะสมรส เป็นคอลัมน์ตัวเลข เพื่อให้โมเดลเข้าใจความแตกต่างโดยไม่มีลำดับสูงต่ำ



BEFORE

marital
'married'
'single'



AFTER

marital_married	marital_single
1	0
0	1

การเตรียมข้อมูล : แปลง 'คำ' ให้เป็น 'ตัวเลข'

```
binary_cols = ['default', 'housing', 'loan', 'y']
for col in binary_cols:
    df[col] = df[col].map({'yes': 1, 'no': 0})

cat_cols = ['job', 'marital', 'education', 'contact', 'month', 'poutcome']
le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

df

แปลงตัวแปรเป้าหมาย
'y' (Target) จากคำว่า
'yes' หรือ 'no' แทนด้วย
yes = 1 และ no = 0

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	4	1	2	0	2143	1	0	2	5	8	261	1	-1	0	3	0
1	44	9	2	1	0	29	1	0	2	5	8	151	1	-1	0	3	0
2	33	2	1	1	0	2	1	1	2	5	8	76	1	-1	0	3	0
3	47	1	1	3	0	1506	1	0	2	5	8	92	1	-1	0	3	0
4	33	11	2	3	0	1	0	0	2	5	8	198	1	-1	0	3	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	9	1	2	0	825	0	0	0	17	9	977	3	-1	0	3	1
45207	71	5	0	0	0	1729	0	0	0	17	9	456	2	-1	0	3	1
45208	72	5	1	1	0	5715	0	0	0	17	9	1127	5	184	3	2	1
45209	57	1	1	1	0	668	0	0	1	17	9	508	4	-1	0	3	0
45210	37	2	1	1	0	2971	0	0	0	17	9	361	2	188	11	1	0

45211 rows × 17 columns

การแบ่งข้อมูล (TRAIN-TEST SPLIT)



จากการสำรวจข้อมูลเบื้องต้น จะเห็นว่ามี
ความไม่สมดุล เพราะสัดส่วนในการตอบรับ
“Yes” น้อยมาก นั่นแสดงว่า โมเดลเรียนรู้ที่
จะตอบ “No”

แบ่งข้อมูลออกเป็น 2 ส่วน คือ

- ชุดสำหรับสอนโมเดล (Training Set) 80%
- ชุดสำหรับทดสอบ (Test Set) 20%

```
from sklearn.model_selection import train_test_split

train_data, test_data = train_test_split(
    df,
    test_size=0.30,
    random_state=8717,
    stratify=df["y"]
)

print("train:", train_data.shape, "test:", test_data.shape)
```

train: (31647, 17) test: (13564, 17)



- **Training Set:** มีจำนวนข้อมูลทั้งหมด 31,647 รายการ สำหรับการสร้างแบบจำลอง.
- **Test Set:** มีจำนวนข้อมูลทั้งหมด 13,564 รายการ สำหรับการประเมินความสามารถในการทำนาย (Generalization).
- **Data Dimensionality:** ข้อมูลทั้งสองส่วนยังคงรักษาตัวแปรต้น (Features) ไว้ที่ 17 ตัวแปร เท่าเดิม.
- **Reproducibility:** มีการกำหนดค่า Random State เพื่อให้กระบวนการแบ่งข้อมูลสามารถตรวจสอบซ้ำและให้ผลลัพธ์ที่คงเดิมได้.

RESULT : TRAIN-TEST- ORIGINAL

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lightgbm	Light Gradient Boosting Machine	0.9058	0.9324	0.9058	0.8974	0.9001	0.4909	0.4970	1.4080
gbc	Gradient Boosting Classifier	0.9055	0.9241	0.9055	0.8937	0.8956	0.4521	0.4684	2.8620
xgboost	Extreme Gradient Boosting	0.9033	0.9261	0.9033	0.8958	0.8986	0.4873	0.4915	0.5780
lr	Logistic Regression	0.9003	0.9017	0.9003	0.8850	0.8861	0.3904	0.4159	2.3540
rf	Random Forest Classifier	0.9001	0.9244	0.9001	0.8851	0.8865	0.3945	0.4183	2.2650
ada	Ada Boost Classifier	0.9000	0.9086	0.9000	0.8868	0.8897	0.4220	0.4366	0.9720
ridge	Ridge Classifier	0.8991	0.9068	0.8991	0.8827	0.8796	0.3413	0.3839	0.3030
lda	Linear Discriminant Analysis	0.8991	0.9068	0.8991	0.8882	0.8916	0.4425	0.4506	0.3480
et	Extra Trees Classifier	0.8968	0.9077	0.8968	0.8802	0.8822	0.3698	0.3941	2.3850
dummy	Dummy Classifier	0.8830	0.5000	0.8830	0.7798	0.8282	0.0000	0.0000	0.2930
knn	K Neighbors Classifier	0.8824	0.7600	0.8824	0.8609	0.8672	0.2939	0.3098	0.7680
dt	Decision Tree Classifier	0.8747	0.6988	0.8747	0.8753	0.8749	0.3959	0.3960	0.4420
nb	Naive Bayes	0.8557	0.8217	0.8557	0.8725	0.8630	0.3750	0.3789	0.3330
qda	Quadratic Discriminant Analysis	0.7879	0.7705	0.7879	0.8646	0.8035	0.3064	0.3203	0.3240
svm	SVM - Linear Kernel	0.7553	0.5939	0.7553	0.8326	0.7585	0.1025	0.1310	0.5120

จากการแบ่งข้อมูล ยังไม่ใช้ SMOTE พบว่า LightGBM ให้ผลดีที่สุด โดยมี Accuracy ~90.6% และ AUC ~0.9324 แสดงว่าโมเดลสามารถแยกกลุ่มลูกค้าที่จะฝากเงินได้ดี และมีความสมดุลของ Precision-Recall สูง (F1-Score ~0.9001) จึงเลือกเป็น baseline model ก่อนทำ imbalance handling

การแบ่งข้อมูล (TRAIN-TEST SPLIT) - SMOTE

ใช้ เทคนิค Stratify และ SMOTE เพื่อรักษาคุณภาพของสัดส่วนคลาส Yes/No ให้เท่ากันทั้งสองชุด ป้องกันความลำเอียง (Bias)

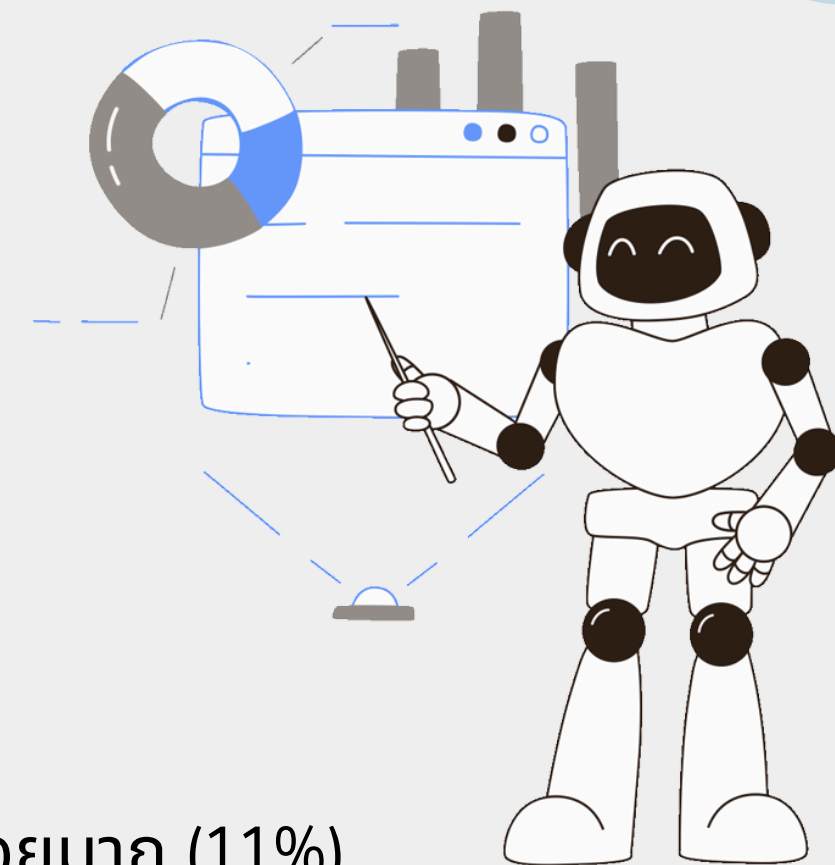
- **Stratify**: คือการ "จัดสรร" (แบ่งเล็กให้เท่ากันทุกงาน) ใช้ตอนแบ่งข้อมูล Train/Test
- **SMOTE**: คือการ "สร้างเพิ่ม" (ทำเค้กจำลองขึ้นมา) ใช้เฉพาะกับข้อมูลฝั่ง Train เพื่อให้ AI เรียนรู้ได้ดีขึ้น

Step 1 : แบ่งข้อมูลแบบ Stratify เพื่อรักษาสัดส่วนกลุ่มตัวอย่าง

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
print(f"สัดส่วน Yes ก่อนทำ SMOTE: {y_train.mean():.2%}")
```

สัดส่วน Yes ก่อนทำ SMOTE: 11.70%

เนื่องจากข้อมูลลูกค้าที่สมัคร (Yes) มีน้อยมาก (11%) การสุ่มแบ่งข้อมูลแบบปกติอาจทำให้กลุ่ม Yes ในชุด Test น้อยเกินไปจนวัดผลไม่ได้ จึงใช้ Stratify เพื่อล็อกสัดส่วนให้ตรงตามความเป็นจริงของ Dataset ทั้งในชุดฝึกสอนและชุดทดสอบ



การแบ่งข้อมูล (TRAIN-TEST SPLIT) - SMOTE

Step 2 : ใช้ SMOTE กับข้อมูล Train เท่านั้น เพื่อสร้างข้อมูลจำลองกลุ่ม Yes ให้เท่ากับกลุ่ม No ในชุดฝึกสอน

```
sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)

print(f"สัดส่วน Yes หลังทำ SMOTE: {y_train_res.mean():.2%}")
```

สัดส่วน Yes หลังทำ SMOTE: 50.00%

ใช้ SMOTE เพื่อแก้ปัญหา Data Imbalance โดยสร้างข้อมูลกลุ่ม Yes จำลองขึ้นมา เฉพาะในชุดฝึกสอน (Train Set) เพื่อให้ Model ได้เรียนรู้ลักษณะของคนที่จะสมัครมากขึ้น แต่เราจะไม่ทำในชุด Test เพราะชุด Test ต้องเป็นข้อมูลดิบที่สะอาดที่สุดเพื่อสะท้อนประสิทธิภาพจริงของ Model

Step 3: Train Model

```
# Step 3: เทรนโมเดล
model.fit(X_train_resampled, y_train_resampled)
```

RandomForestClassifier

```
RandomForestClassifier(max_depth=10, random_state=42)
```

มีการใช้ข้อมูลที่ผ่านการทำ Resampling มาเทรนโมเดล เพื่อแก้ปัญหาที่กลุ่มลูกค้าเป้าหมาย (Class 1) มีจำนวนน้อยกว่าปกติมาก

การแบ่งข้อมูล (TRAIN-TEST SPLIT) - SMOTE

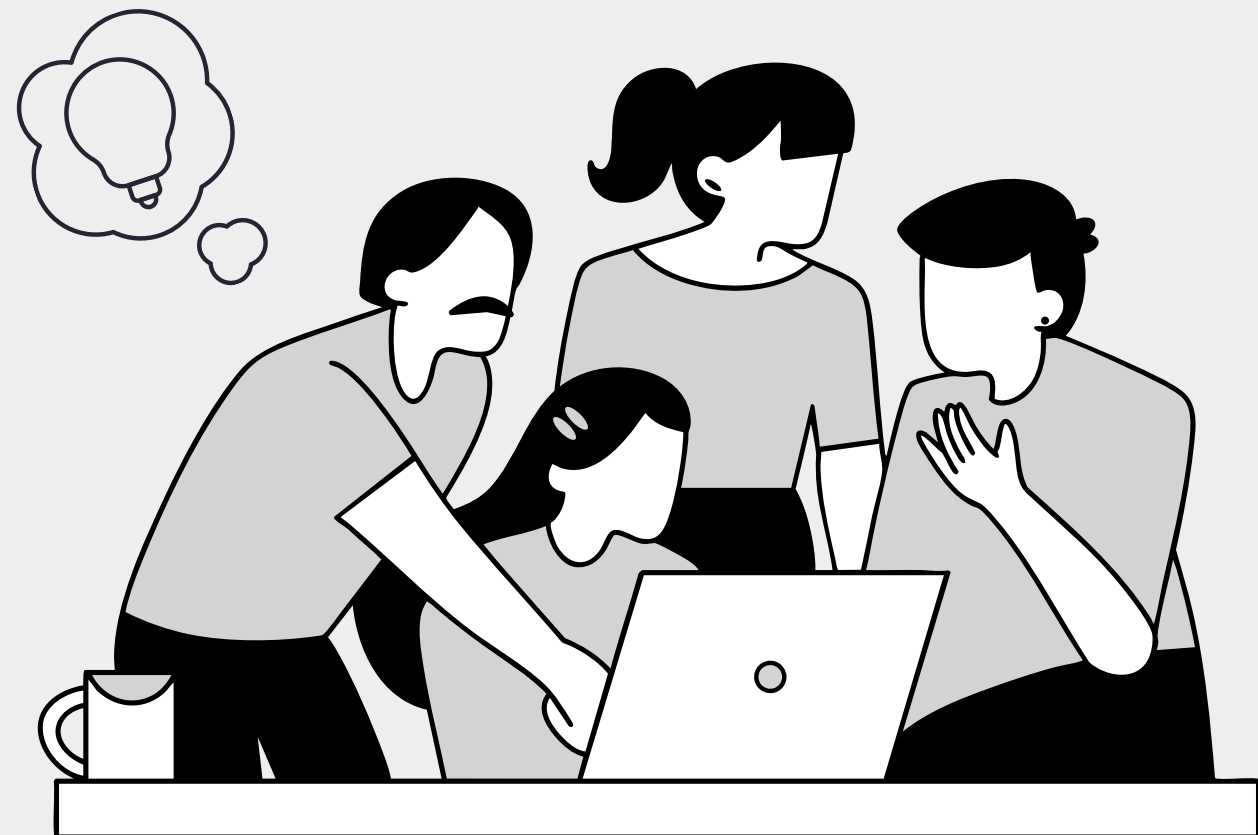
Step 4 : การวัดผลและการใช้ SMOTE

```
# Step 4: สอบวัดผล
y_pred = model.predict(X_test)

print("\n--- After SMOTE Result ---")
print(y_train_smote.value_counts())
```

```
--- After SMOTE Result ---
y
0    31937
1    31937
Name: count, dtype: int64
```

ผลลัพธ์ในส่วน After SMOTE Result แสดงให้เห็นว่าเราปรับสมดุลข้อมูลจนคลาส 0 และ 1 มีจำนวนเท่ากันที่ 31,937 รายการ ซึ่งช่วยให้โมเดลไม่ลำเอียง (Bias) ไปทางคลาสใดคลาสหนึ่ง



RESULT : TRAIN-TEST (SMOTE)

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
nb	Naive Bayes	0.8557	0.8217	0.5191	0.4086	0.4568	0.3750	0.3789	0.4590
qda	Quadratic Discriminant Analysis	0.7879	0.7705	0.5097	0.3845	0.4015	0.3064	0.3203	1.0520
xgboost	Extreme Gradient Boosting	0.9033	0.9261	0.4859	0.6099	0.5405	0.4873	0.4915	0.9510
lightgbm	Light Gradient Boosting Machine	0.9058	0.9324	0.4767	0.6297	0.5423	0.4909	0.4970	1.2180
dt	Decision Tree Classifier	0.8747	0.6988	0.4693	0.4649	0.4669	0.3959	0.3960	0.5990
lda	Linear Discriminant Analysis	0.8991	0.9068	0.4261	0.5972	0.4968	0.4425	0.4506	0.6310
gbc	Gradient Boosting Classifier	0.9055	0.9241	0.4052	0.6573	0.5010	0.4521	0.4684	4.2100
ada	Ada Boost Classifier	0.9000	0.9086	0.3852	0.6175	0.4740	0.4220	0.4366	1.4840
rf	Random Forest Classifier	0.9001	0.9244	0.3408	0.6385	0.4438	0.3945	0.4183	3.9690
lr	Logistic Regression	0.9003	0.9017	0.3342	0.6425	0.4391	0.3904	0.4159	5.0430
et	Extra Trees Classifier	0.8968	0.9077	0.3207	0.6149	0.4203	0.3698	0.3941	3.5790
svm	SVM - Linear Kernel	0.7553	0.5939	0.3154	0.2836	0.1969	0.1025	0.1310	0.7140
knn	K Neighbors Classifier	0.8824	0.7600	0.2744	0.4957	0.3527	0.2939	0.3098	1.1610
ridge	Ridge Classifier	0.8991	0.9068	0.2709	0.6726	0.3855	0.3413	0.3839	0.4200
dummy	Dummy Classifier	0.8830	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3880

- **โมเดลประสิทธิภาพสูงสุด:** LightGBM อันดับ 1 ในทุกมิติสำคัญ ทั้งความแม่นยำ (Accuracy 0.9076), การจำแนกคลาส (AUC 0.9324) และความสมดุลของผลพยากรณ์ (F1-Score 0.9024)
- **คุณภาพข้อมูล:** ทุกโมเดลทำค่า AUC ได้สูงกว่า 0.90 สะท้อนว่าคุณลักษณะข้อมูล (Features) มีประสิทธิภาพในการจำแนกสูงมาก



COMPARE : ORIGINAL & SMOTE

ORIGINAL

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)	
	lightgbm	Light Gradient Boosting Machine	0.9072	0.9329	0.9072	0.8990	0.9017	0.4998	0.5055	2.4550
	xgboost	Extreme Gradient Boosting	0.9024	0.9256	0.9024	0.8940	0.8970	0.4771	0.4819	0.9490
	lr	Logistic Regression	0.9017	0.9007	0.9017	0.8870	0.8879	0.4008	0.4260	2.6140

โมเดลประสิทธิภาพสูงสุด คือ LightGBM เป็นอันดับ 1 ในทุกมิติสำคัญ ทั้งในข้อมูลแบบ Original และ แบบที่ทำการ SMOTE ซึ่งมีค่าผลลัพธ์ของค่าต่างๆที่ไม่แตกต่างกันมากนัก

SMOTE

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)	
	lightgbm	Light Gradient Boosting Machine	0.9076	0.9314	0.9076	0.8998	0.9024	0.5043	0.5095	5.5350
	xgboost	Extreme Gradient Boosting	0.9041	0.9268	0.9041	0.8975	0.9001	0.4980	0.5008	2.5820
	lr	Logistic Regression	0.8406	0.9034	0.8406	0.9045	0.8611	0.4574	0.4959	4.7160

บทสรุป : พิจารณาเลือกใช้ **“LightGBM” แบบ Original** เป็นโมเดลหลักเนื่องจากให้ความแม่นยำและค่าความสมดุล (F1-Score) ดีที่สุด และการใช้ข้อมูลแบบไม่ SMOTE จะลดความ Bias ของข้อมูลที่สังเคราะห์ขึ้นมา



AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

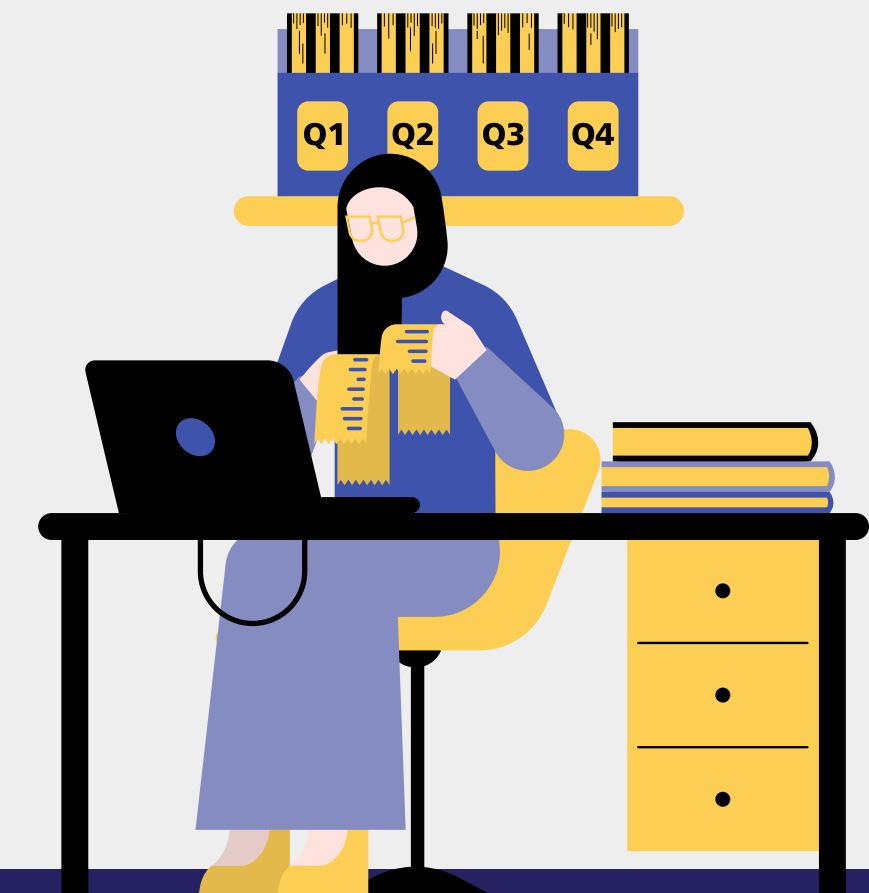
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset



CLASSIFICATION MODEL

```
from sklearn.model_selection import train_test_split
train_data, test_data = train_test_split(df, test_size=0.3,
                                         random_state=42,
                                         stratify=df['y'])
```

```
from pycaret.classification import *
# Setup classification experiment
clf1 = setup(session_id=8717, data=train_data, target='y')
# Compare classification models
models = compare_models(sort='Recall')
```

Run PyCaret with Train Set

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lightgbm	Light Gradient Boosting Machine	0.9058	0.9324	0.9058	0.8974	0.9001	0.4909	0.4970	1.4080
gbc	Gradient Boosting Classifier	0.9055	0.9241	0.9055	0.8937	0.8956	0.4521	0.4684	2.8620
xgboost	Extreme Gradient Boosting	0.9033	0.9261	0.9033	0.8958	0.8986	0.4873	0.4915	0.5780
lr	Logistic Regression	0.9003	0.9017	0.9003	0.8850	0.8861	0.3904	0.4159	2.3540
rf	Random Forest Classifier	0.9001	0.9244	0.9001	0.8851	0.8865	0.3945	0.4183	2.2650
ada	Ada Boost Classifier	0.9000	0.9086	0.9000	0.8868	0.8897	0.4220	0.4366	0.9720
ridge	Ridge Classifier	0.8991	0.9068	0.8991	0.8827	0.8796	0.3413	0.3839	0.3030
lda	Linear Discriminant Analysis	0.8991	0.9068	0.8991	0.8882	0.8916	0.4425	0.4506	0.3480
et	Extra Trees Classifier	0.8968	0.9077	0.8968	0.8802	0.8822	0.3698	0.3941	2.3850
dummy	Dummy Classifier	0.8830	0.5000	0.8830	0.7798	0.8282	0.0000	0.0000	0.2930
knn	K Neighbors Classifier	0.8824	0.7600	0.8824	0.8609	0.8672	0.2939	0.3098	0.7680
dt	Decision Tree Classifier	0.8747	0.6988	0.8747	0.8753	0.8749	0.3959	0.3960	0.4420
nb	Naive Bayes	0.8557	0.8217	0.8557	0.8725	0.8630	0.3750	0.3789	0.3330
qda	Quadratic Discriminant Analysis	0.7879	0.7705	0.7879	0.8646	0.8035	0.3064	0.3203	0.3240
svm	SVM - Linear Kernel	0.7553	0.5939	0.7553	0.8326	0.7585	0.1025	0.1310	0.5120

Model Performance Comparison Top 3 models

Model	เหตุผลที่เลือก
LightGBM	ให้ผลดีที่สุด
XGBoost	เปรียบเทียบ boosting
Logistic Regression	Baseline เชิงเส้น



ไม่เลือก GBC (Gradient Boosting Classifier)
เพราะ LightGBM และ XGBoost ก็เป็น
boosting เหมือนกันและใหม่กว่า ทำให้ GBC
ไม่เพิ่ม insight ใหม่

CLASSIFICATION MODEL

สร้าง Model ทั้ง 3 ตัว

```
from pycaret.classification import *  
lgbm = create_model('lightgbm')  
xgb = create_model('xgboost')  
lr = create_model('lr')
```

Run Model with Test Set

	Model	Accuracy	Precision_yes	Recall_yes	F1_yes
0	LightGBM	0.909245	0.648333	0.490233	0.558306
1	XGBoost	0.904748	0.619627	0.481411	0.541844
2	Logistic Regression	0.900840	0.646845	0.335854	0.442140

LightGBM

Recall(yes) = 0.49

ช่วยลดการ “พลาดลูกค้าที่ควรโทร” ได้มากที่สุด

XGBoost

Recall(yes) = 0.48

ประสิทธิภาพดี แต่ด้อยกว่า LightGBM เล็กน้อยในด้านการจับลูกค้าเป้าหมาย

Logistic Regression

Recall(yes) = 0.33

แม้ Precision และ Accuracy ไม่แย่ แต่พลาดลูกค้าที่สมัครจริงจำนวนมาก

“เลือกใช้โมเดล LightGBM เป็นโมเดลสุดท้าย เนื่องจากสามารถให้ค่า Recall ของกลุ่มลูกค้าที่สมัครได้สูงที่สุด ซึ่งเป็นปัจจัยสำคัญอย่างยิ่งสำหรับปัญหาทางธุรกิจนี้”

CLASSIFICATION MODEL : LIGHTGBM

Run Model with Test Set

LGBMClassifier Confusion Matrix

True Class	0	1
	Predicted Class	Predicted Class
0	8067 True Negative (TN)	317 False Positive (FP)
1	530 False Negative (FN)	581 True Positive (TP)

Classification Report:				
	precision	recall	f1-score	support
no	0.93	0.96	0.95	11977
yes	0.65	0.49	0.56	1587
accuracy			0.91	13564
macro avg	0.79	0.73	0.75	13564
weighted avg	0.90	0.91	0.90	13564

$$Recall = \frac{TP}{TP + FN} = \frac{581}{581 + 530} \approx 0.49$$

$$Precision = \frac{TP}{TP + FP} = \frac{581}{581 + 317} \approx 0.65$$

จากลูกค้าที่สมัครจริงทั้งหมด โมเดลสามารถจับได้ประมาณ **49%**
ลูกค้าที่โมเดลทำนายว่า "สมัคร" ประมาณ **65%** สมัครจริง
จาก Confusion Matrix แสดงให้เห็นว่า LightGBM สามารถระบุ
ลูกค้าที่ไม่สมัครได้ดีมาก แต่ยังมีโอกาสพัฒนาต่อเพื่อเพิ่มการจับ
ลูกค้าที่สมัครจริง โดยเฉพาะการลดจำนวน False Negative



CLASSIFICATION MODEL : LIGHTGBM

Save Best Model

```
best_model = lgbm
```

```
save_model(best_model, 'Bank_Term_Deposit_LightGBM_Final')
```

นำ Model มาทดสอบกับข้อมูล test_y.csv

```
from pycaret.classification import *  
from sklearn.metrics import confusion_matrix, classification_report  
import pandas as pd
```

```
final_model = load_model('Bank_Term_Deposit_LightGBM_Final')  
eval_df = pd.read_csv('test_y.csv')  
out = predict_model(final_model, data=eval_df)  
y_eval = eval_df['y']
```

```
# ประเมินผล  
print(classification_report(y_eval, out['prediction_label']))  
print(confusion_matrix(y_eval, out['prediction_label']))
```

Transformation Pipeline and Model Successfully Loaded

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

no	0.95	0.97	0.96	4000
yes	0.72	0.58	0.64	521
accuracy			0.93	4521
macro avg	0.83	0.78	0.80	4521
weighted avg	0.92	0.93	0.92	4521

```
[[3883  117]  
 [ 218  303]]
```



จากการนำ Model มาทดสอบกับข้อมูลอีกชุด ที่ Model ไม่เคยเห็น จะเห็นได้ว่า Model สามารถจับลูกคำที่สมัครได้มากกว่าเดิม จากลูกคำที่ Model ทำนายว่า “สมัคร” มี **72%** ที่สมัครจริง และลูกคำที่สมัครจริงทั้งหมด Model จับได้ถึง **58%**

CLASSIFICATION MODEL : LIGHTGBM

การนำ Model ไปใช้งานจริง

```
test_predictions = predict_model(final_model, data=test_df)
test_predictions.head()
```

```
# เลือกลูกค้าที่มีโอกาสสูง
high_potential = test_predictions[
    (test_predictions['prediction_label'] == 'yes') &
    (test_predictions['prediction_score'] >= 0.7)
]

high_potential.head()
```



	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	prediction_label	prediction_score
30	68	retired	divorced	secondary	no	4189	no	no	telephone	14	jul	897	2	-1	0	unknown	yes	0.7318
40	38	management	single	tertiary	no	11971	yes	no	unknown	17	nov	609	2	101	3	failure	yes	0.7402
49	61	admin.	married	unknown	no	4629	yes	no	cellular	27	jan	181	1	92	1	success	yes	0.7835
70	37	management	married	tertiary	no	0	no	no	cellular	16	jul	268	2	182	3	success	yes	0.8642
98	36	blue-collar	divorced	secondary	no	2843	no	no	cellular	12	feb	473	1	182	1	success	yes	0.8782

```
high_potential.to_csv('Target_Customers_Term_Deposit.csv', index=False)
```

เพิ่มคอลัมน์:

- prediction_label → yes / no
- prediction_score → ความน่าจะเป็น (probability)

AGENDA

1

Pain Point/ Motivation/
Challenge/ Benefits

2

Dataset Understanding/
Data Exploration

3

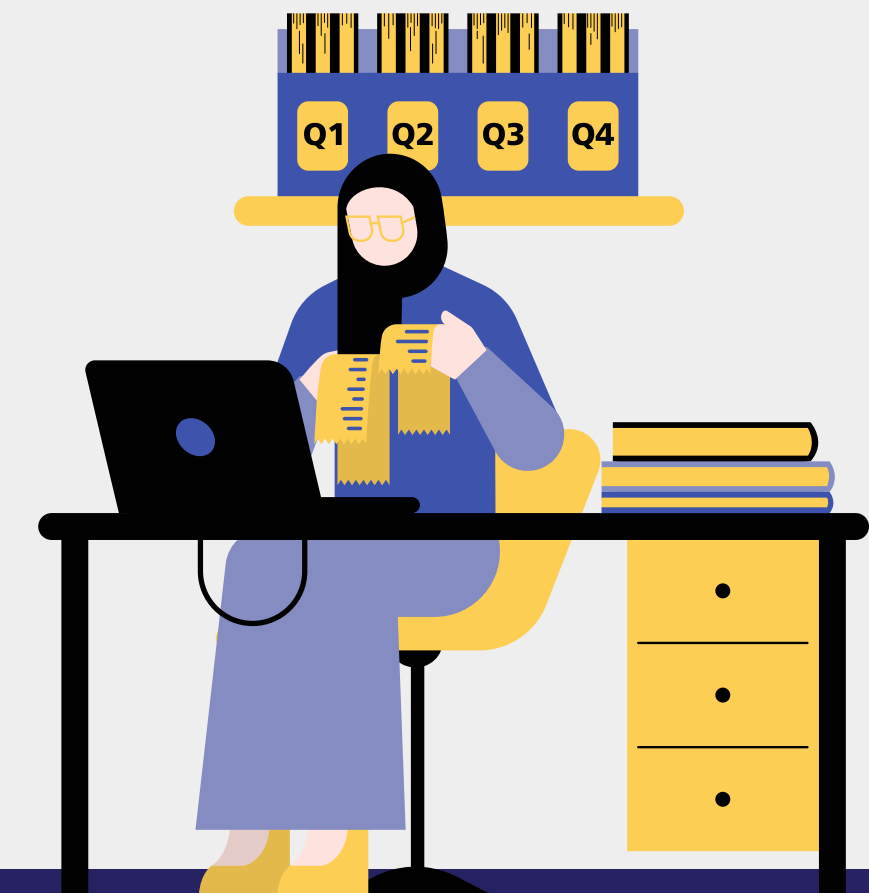
Data Preprocessing

4

Modeling and
Performance of
the Model

5

Insight from Dataset

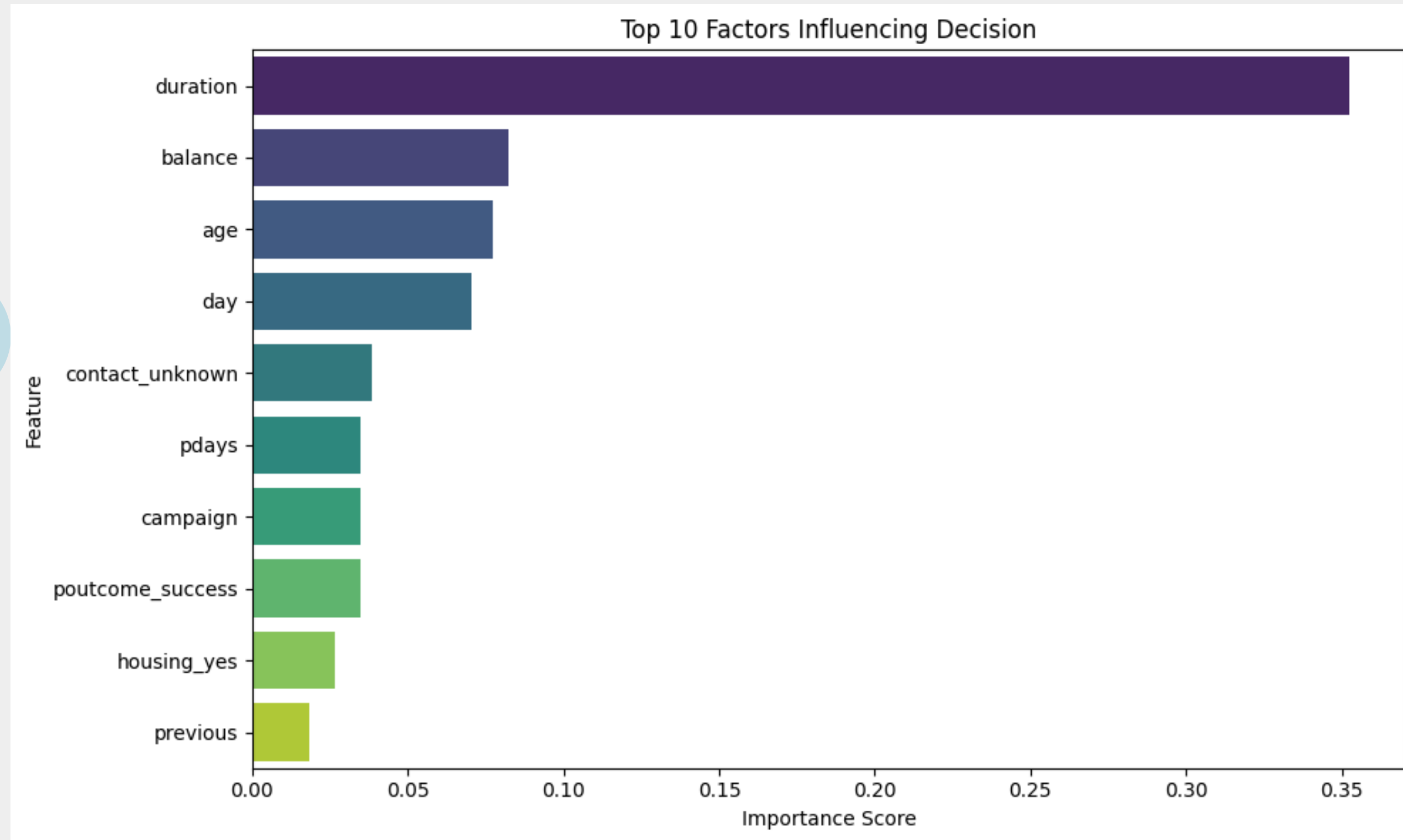


ข้อมูลเชิงลึก (INSIGHTS)

จากการวิเคราะห์ปัจจัยที่มีผลต่อโมเดล (Feature Importance) พบข้อมูลเชิงลึกที่สำคัญทางธุรกิจ ดังนี้:

- 1 **Duration (ระยะเวลาการสนทนา):** คุณภาพสำคัญกว่าปริมาณ ซึ่งเป็นปัจจัยที่มีค่าน้ำหนักสูงสุดอย่างต่อเนื่อง
- 2 **Poutcome (ผลลัพธ์ในอดีต):** ลูกค้าที่เคยสมัคร (Success) ในอดีต มีแนวโน้มจะสมัครซ้ำสูงกว่าลูกค้าใหม่มาก
- 3 **Month (เดือนที่ติดต่อ) :** ข้อมูลมักชี้ให้เห็นว่าบางเดือน (เช่น กันยายน, มีนาคม และ ตุลาคม) มีอัตราการสมัครสูงกว่าปกติ ธนาคารควรจัดสรรงบประมาณการตลาดและพนักงานโทรในช่วงเดือนที่สถิติชี้ว่าคนพร้อมออมเงินที่สุด (Seasonal Marketing)
- 4 **Balance (ยอดเงินในบัญชี):** ยอดเงินไม่ได้บอกแค่ว่าเขารวยไหม แต่บอกว่าเขา "มีเงินเย็น" พร้อมออมเพิ่มหรือไม่ ใช้ยอดเงินในการคัดกรอง (Segment) ลูกค้าเพื่อเสนอผลิตภัณฑ์ที่เหมาะสม เช่น คนที่มีเงินเยอะอาจเสนอเงินฝากประจำระยะยาวเพื่อผลตอบแทนที่สูงกว่า
- 5 **Job (อาชีพ) :** มีความสำคัญอย่างยิ่งในขั้นเตรียมการ (Targeting) แต่ในเชิงโมเดลที่ความสำคัญของ Job อยู่อันดับรองลงมา เป็นเพราะตัวแปรอื่นอย่าง Duration และ Poutcome มีพลังในการพยากรณ์ที่เด็ดขาดกว่า แต่สามารถนำอาชีพมาใช้เป็นหลักในการทำ Customer Segmentation เพื่อให้พนักงานสามารถนำเสนอแผนการออมที่เหมาะสมกับช่วงชีวิตของลูกค้าแต่ละกลุ่มได้แม่นยำที่สุด

ข้อมูลเชิงลึก : TOP 10 FACTOR INFLUENCING DECISION



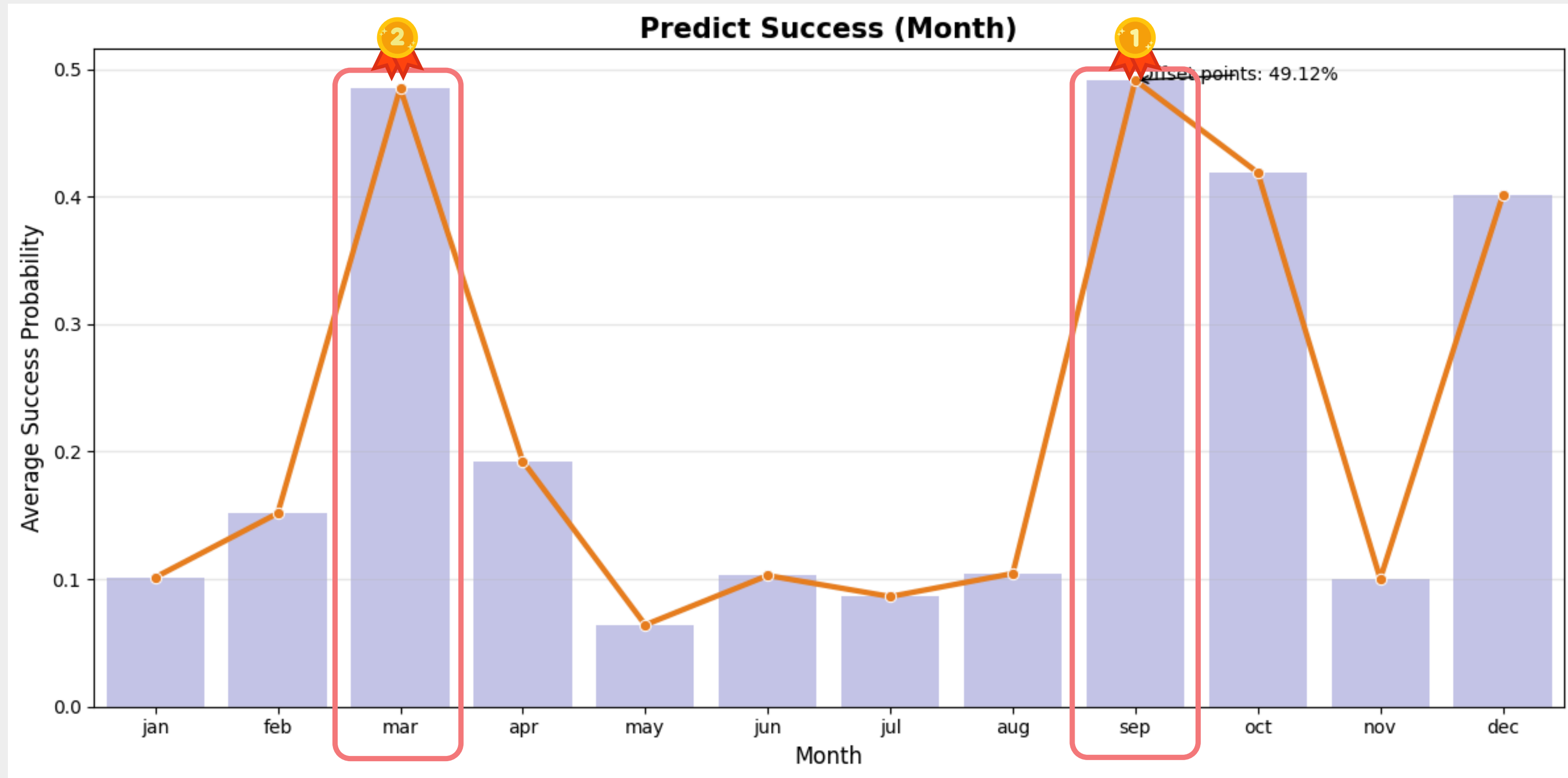
ปัจจัยหลักที่ส่งผลสูงสุด: Duration (ระยะเวลาที่สนทนา):

เป็นตัวแปรที่มีอิทธิพลสูงสุดอย่างชัดเจน (Importance Score ~0.35) แสดงให้เห็นว่า "ความนานในการปฏิสัมพันธ์กับลูกค้า" คือตัวบ่งชี้หลักของความสำเร็จ

ปัจจัยรองที่น่าสนใจ:

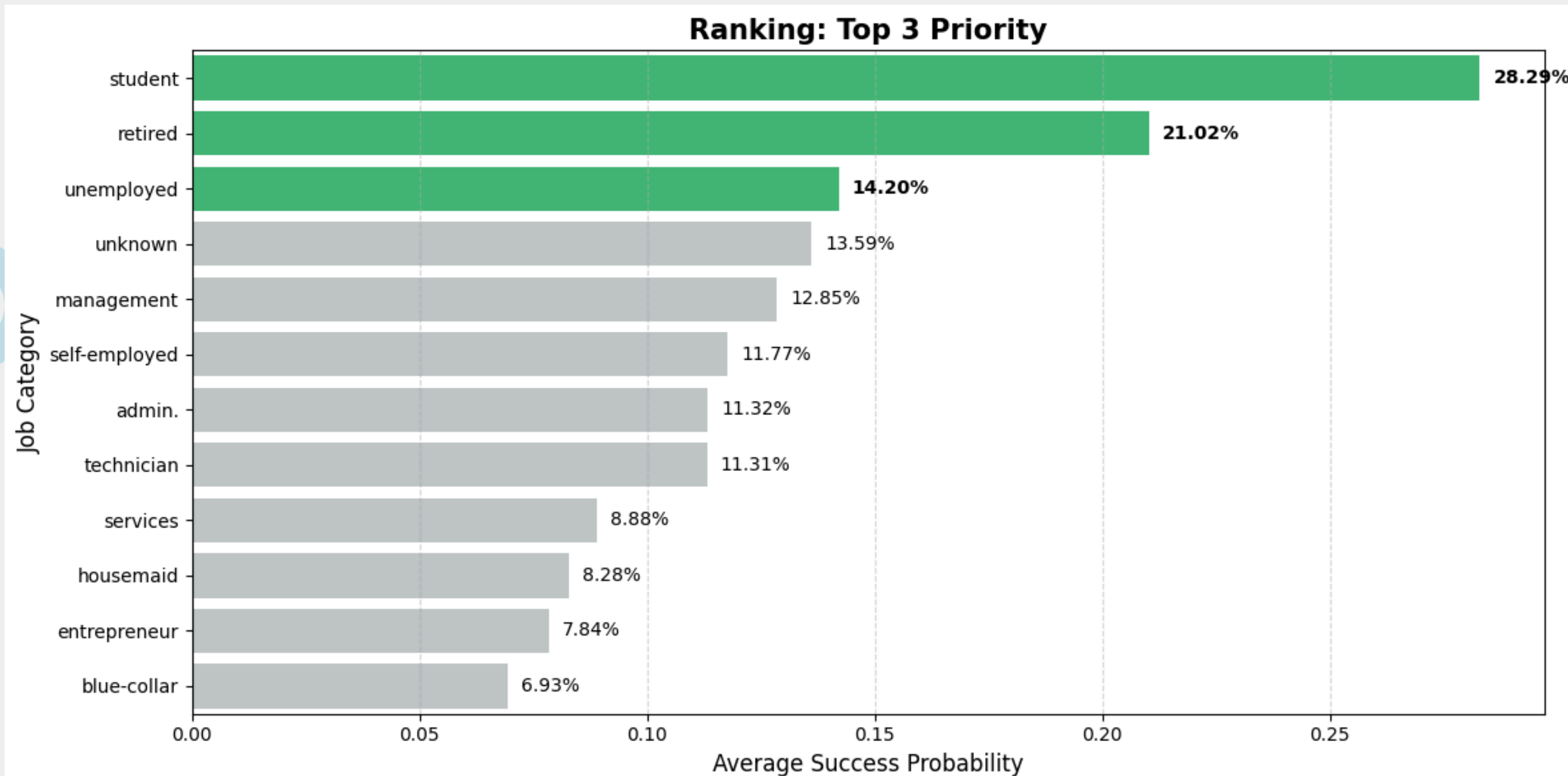
- Balance (ยอดเงินคงเหลือ) และ Age (อายุ): เป็นปัจจัยรองลงมาที่มีผลต่อการตัดสินใจ
- Contact & Campaigns: ข้อมูลการติดต่อในอดีต (pdays, previous) และความสำเร็จของแคมเปญก่อนหน้า (poutcome_success) มีผลต่อโมเดลในระดับใกล้เคียงกัน

ข้อมูลเชิงลึก : OPTIMAL TIMING FOR SUCCESS



เดือนที่โมเดลระบุว่าไตรแล้วมีโอกาสสำเร็จสูงสุดคือ: กันยายน รองมาคือ มีนาคม

ข้อมูลเชิงลึก : TOP RANKING OF JOB CATEGORY



กลุ่มนักเรียนและผู้เกษียณอายุ :
มีความโดดเด่นอย่างมาก ซึ่งอาจสะท้อนว่ากลุ่มที่มี "เวลา" หรือ "ความต้องการวางแผนการเงินในอนาคต/หลังเกษียณ" มีแนวโน้มสนใจแคมเปญเงินฝากนี้มากกว่ากลุ่มวัยทำงานประจำ

กลุ่มอาชีพอื่น ๆ: อาชีพสายบริหาร (Management) หรือสายเทคนิค (Technician) มีโอกาสตอบรับอยู่ในเกณฑ์เฉลี่ยที่ประมาณ 11-12%

กลุ่มที่มีการตอบรับต่ำสุด: คือกลุ่ม Blue-collar (แรงงานฝีมือ/อุตสาหกรรม) ที่ 6.93%

ข้อเสนอแนะ (RECOMMENDATION)

1 ด้านการคัดเลือกกลุ่มเป้าหมาย (Targeting & Lead Scoring)

- เปลี่ยนจากการสุ่มโทร เป็นการโทรตามลำดับความสำคัญ: ธนาคารควรยกเลิกการใช้รายชื่อแบบสุ่ม (Cold Calling) และเปลี่ยนมาใช้ระบบ Lead Scoring จากโมเดล LightGBM โดยให้พนักงานโฟกัสกลุ่มที่มีค่าความน่าจะเป็น (Probability) สูงกว่า 70% ก่อน
- เจาะกลุ่ม "ลูกค้าเก่า" ที่เคยสำเร็จ (Poutcome = Success): ข้อมูลชี้ชัดว่ากลุ่มนี้ปิดการขายได้ง่ายที่สุดและใช้เวลาน้อยที่สุด ควรเป็น Priority อันดับ 1 ในทุกแคมเปญ

2 ด้านช่วงเวลาและจังหวะ (Timing Strategy)

- เน้นเดือนที่มีประสิทธิภาพสูง (Seasonal Marketing): จากการวิเคราะห์ พบว่าเดือนกันยายน, มีนาคม และ ตุลาคม มีโอกาสสำเร็จสูงกว่าปกติ แม้จำนวนคนจะน้อยลง ธนาคารควรจัดโปรโมชั่นที่ "แรง" กว่าเดิมในช่วงเดือนเหล่านี้เพื่อดึงดูดใจ
- ควบคุมจำนวนครั้งที่ติดต่อ (Campaign Frequency): จากผล Correlation ที่ติดลบในตัวแปร Campaign เราแนะนำให้ จำกัดการโทรซ้ำไม่เกิน 3-4 ครั้ง ต่อลูกค้าหนึ่งรายในหนึ่งแคมเปญ เพื่อป้องกันลูกค้าเกิดความรำคาญซึ่งจะส่งผลให้โอกาสสมัครลดลง

3 ด้านทักษะและการสนทนา (Operation & Communication)

- สำหรับ กลุ่ม Management/Retired: ควรเน้นบทสนทนาที่ให้ข้อมูลเชิงลึกและสิทธิประโยชน์ที่ชัดเจน เพราะกลุ่มนี้พร้อมจะฟังนานหากข้อมูลน่าสนใจ
- สำหรับ กลุ่มลูกค้าเก่า: ควรเน้นความรวดเร็ว กระชับ และเสนอโปรโมชั่นทันที (Instant Offer) เพื่อเพิ่มประสิทธิภาพการทำงานของพนักงาน

THANK YOU

